Differential-Linear Attacks from New Distinguishers
the case of SERPENT and PRESENT

Thierno Mamoudou Sabaly[1] and Marine Minier[1]

Université de Lorraine, CNRS, Inria, LORIA, F-54000, Nancy, France,
email: thierno-mamoudou.sabaly@loria.fr, marine.minier@loria.fr

Abstract. Differential-linear distinguishers have been introduced by Langford and Hellman in 1994. They consist in combining, first, a differential distinguisher and second, a linear distinguisher and then study the bias between plaintexts with a difference and linear approximations of the two ciphertexts to create a differential-linear distinguisher. The original method has been improved by Bar-On *et al.* in 2019 where the table called the DLCT (Differential Linear Connectivity Table) has been introduced and more recently, in 2024 by Hadipour *et al.* where, as for the case of boomerang distinguishers, several intermediate tables are used to tune the computation of the middle part of the distinguisher. From a distinguisher, it is thus natural to try to mount some dedicated attacks. This step has been done by Broll *et al.* in 2021 and in 2022 for the case of SERPENT.
In this paper, we propose a tool that directly searches for the best differential-linear attacks automating the work of Broll *et al.* using the differential-linear distinguishers proposed by Hadipour *et al.*. More precisely, both searches (distinguishers and attacks) are done in the same step to improve the overall complexity of the differential-linear attack. We apply this tool to the case of SERPENT and PRESENT. The attack against SERPENT reaches 12 rounds with a time complexity equal to $2^{220.9}$ for a data/memory complexity equal to $2^{125.01}$. The attack against PRESENT-80 (PRESENT-128 respectively) reaches 16 (18 respectively) rounds with a time complexity equal to $2^{73.88}$ (2^{124} respectively) for a data/memory complexity equal to $2^{57.88}$ ($2^{63.25}$ respectively).

Keywords: Block ciphers, Differential-Linear Distinguishers, Differential-Linear Attacks, Automatic Tools

1 Introduction

Differential-linear distinguishers introduced in [LH94] are an extension of both differential and linear distinguishers. Whereas a differential distinguisher aims at finding differential relations between plaintexts and ciphertexts of a cipher that happen with high probability, a linear distinguisher aims at finding linear

relations between plaintexts and ciphertexts of a cipher also with high correlation. In a differential-linear distinguisher, an adversary splits the cipher E into two parts $E = E_1 \circ E_0$ and searches for a differential relation on E_0 of probability p and a linear relation on E_1 with correlation q. Thus, the probability between the inputs and the outputs of the differential-linear distinguisher could be approximated by a correlation pq^2.

The main principles of differential-linear distinguishers have been significantly refined in [BDKW19, HDE24, BDK03, BLN15, BCD+22]. In [BDKW19], the notion of the Differential-Linear Connectivity Table (DLCT) is introduced to better capture the correlation of the middle rounds. In [HDE24], based on what have been done for boomerang distinguishers, several new tables suited for a better computation of the transition between the differential and linear parts are introduced. With these new tools, the adversary no longer divides the cipher into two parts but into three, with E decomposed as $E = E_1 \circ E_m \circ E_0$. Here, the differential behavior is studied in E_0 with probability p, the linear behavior in E_1 with correlation q, and the transition between the differential and linear worlds is handled in E_m with correlation r. Thus, the overall correlation of the differential-linear distinguisher could be approximated by prq^2. [HDE24] generalizes the DLCT framework and allows the improvement of previous differential-linear distinguishers for various ciphers. In [BCD+22], the computation of differential-linear biases are revised and provides more precise formulas for estimating the complexities of DL attacks. In [BDK03, BLN15], DL attacks are precisely analyzed with particular techniques to enhance key guessing strategies.

In this paper, we extend the tool developed by Hadipour *et al.* [HDE24] to compute not only differential-linear distinguishers but complete differential-linear attacks. The original tool[1] was limited to finding distinguishers; we modified it[2] so that it now searches directly for full differential-linear attacks. Indeed, the best distinguisher does not necessarily lead to the best attack. Therefore, the search for differential-linear attacks must be carried out in a unified manner to ensure the discovery of the most effective attack. For example, when running our tool configured to follow the best distinguishers identified in [HDE24] for SERPENT, we were unable to derive a viable 12-round attack. However, our enhanced tool identified an alternative distinguisher—although with lower correlation—that leads to the successful attack presented in this paper. We also applied our tool to the PRESENT block cipher. Our new results are summarized in Table 1.

This paper is organized as follows: Section 2 describes all the required materials to understand differential-linear attacks; Section 3 presents our approach and how we tune the tool provided by Hadipour *et al.* to mount automated attacks and no more automated distinguishers; Section 4 recalls the ciphers under

[1] The original tool is available at the following link: `https://github.com/hadipourh/DL`.

[2] Our tool is available at the following link: `https://gitlab.inria.fr/tsabaly/dl_keyrtool`.

Table 1: Comparison of selected differential-linear and related attacks on SERPENT and PRESENT.

Ciphers	Attack Type	Attack nb rnds	Dist. nb rnds	Time	Data	Memory	Success Proba.	Sources
SERPENT 256	DL[a]	12	9	2^{251}	2^{127}	2^{127}	-	[LLL21]
	MD. Lin.[a b]	12	-	2^{242}	$2^{125.8}$	2^{236}	-	[NWW11][BCD+22][c]
	DL	12	9	$2^{233.55}$	$2^{127.92}$	$2^{127.92}$	0.1	[BCD+22]
	DL	12	9	$2^{242.93}$	$2^{118.40}$	$2^{118.40}$	0.1	[BCD+22]
	Lin‡	12	9	$2^{214.36}$	$2^{125.16}$	$2^{125.16}$	0.81	[FGT24]
	Lin‡	12	9	$2^{210.36}$	$2^{126.3}$	$2^{125.16}$	0.80	[FGT24]
	DL	12	9	$\mathbf{2^{219.77}}$	$\mathbf{2^{118.67}}$	$\mathbf{2^{118.67}}$	0.1	Section 4.1
	DL	12	9	$\mathbf{2^{238.38}}$	$\mathbf{2^{110.41}}$	$\mathbf{2^{110.41}}$	0.1	Section 4.1
	DL	12	9	$\mathbf{2^{220.79}}$	$\mathbf{2^{125.01}}$	$\mathbf{2^{125.01}}$	0.85	Section 4.1
PRESENT 80	Lin.[a]	**29**	24	$2^{78.87}$	$2^{63.93}$	2^{71}	0.512	[WLW24]
	Diff.[a]	18	14	2^{59}	2^{58}	2^{59}	-	[BDD+24]
	DL	16	13	$2^{73.88}$	$2^{57.88}$	$2^{57.88}$	0.1	Section 4.2
	DL	16	13	$2^{79.91}$	$2^{63.91}$	$2^{63.91}$	0.825	Section 4.2
PRESENT 128	Lin.	**29**	24	$2^{126.33}$	$2^{62.88}$	$2^{97.91}$	0.628	[WLW24]
	DL	18	13	2^{124}	$2^{63.25}$	$2^{63.25}$	0.1	Section 4.2

[a] MD. Lin: Multidimensional Linear — Lin.: Linear — Diff.: Differential — DL: Differential-Linear

[b] MD. Lin attack is a 56-dimensional linear cryptanalysis.

[c] [NWW11] introduced the attack and [BCD+22] refined the complexity estimates.

‡ Linear attacks start at round 2 (i.e S-box S_2) instead of first round (i.e S-box S_0).

study (SERPENT and PRESENT) and gives the differential-linear attacks we found; finally, Section 5 concludes this paper.

2 Related Work

2.1 Differential-Linear (DL) Cryptanalysis

In this section, we present an overview of differential-linear cryptanalysis. We begin by introducing the distinguishers that constitute its building blocks. Recall that a difference Δ corresponds to the XOR of two messages while a mask λ indicates the linear relations in a message. Then for a permutation Π, a *differential trail* is defined by an input difference Δ_i and an output difference Δ_o that occurs with a probability $p = \mathbb{P}(\Pi(P_1) \oplus \Pi(P_2) = \Delta_o | P_1 \oplus P_2 = \Delta_i)$, we denote $\Delta_i \xrightarrow{p} \Delta_o$. And a *linear trail* is defined by an input mask λ_i, an output mask λ_o and a correlation $q = 2\mathbb{P}(\lambda_i \cdot P = \lambda_o \cdot \Pi(P)) - 1$, we denote $\lambda_i \xrightarrow{q} \lambda_o$.

Differential cryptanalysis exploits differential characteristics with high probability to distinguish a block cipher from a random permutation. This can lead to different security breaks as a key recovery (full or partial) or the possibility

of creating MAC collisions. This attack was introduced by Biham and Shamir [BS91] in 1990.

Linear cryptanalysis was introduced by Matsui [Mat94] in 1994. This cryptanalysis exploits linear approximations with correlation q such that $|q|$ is significantly larger than 0. These relations arise from inherent biases introduced by the non-linear components of the permutation. This cryptanalysis leads to the ability of distinguishing a block cipher from a random permutation and also the possibility of a full or partial key recovery, as it has been done for the DES [Mat94] case.

Symmetric block ciphers are successive applications of a same permutation called a round function. For a small number of rounds, differential and linear attacks are efficient as one can find good trails. But if the number of rounds increases then the probability of the trails decreases. This leads to the introduction of the differential-linear cryptanalysis by Langford and Hellman [LH94] in 1994. This approach is the combination of a short differential characteristic followed by a short linear characteristic giving a larger characteristic called *differential-linear characteristic*.

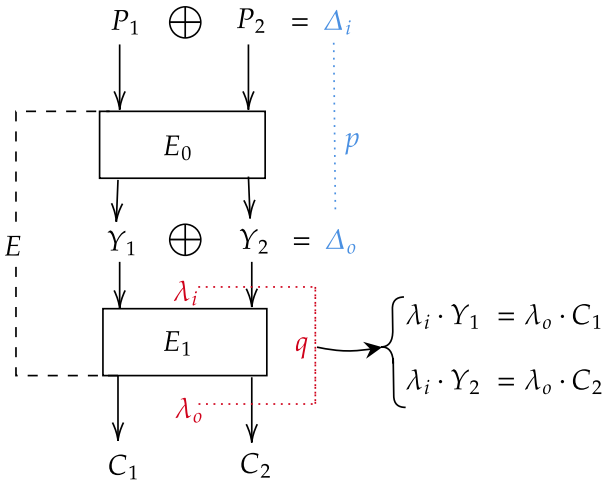


Fig. 1: Construction of a differential-linear distinguisher.

More formally, the block cipher E is split, as shown in Figure 1, in two parts E_0 with a differential trail $\Delta_i \xrightarrow{p} \Delta_o$ and E_1 with a linear trail $\lambda_i \xrightarrow{q} \lambda_o$ such that $E = E_1 \circ E_0$. To distinguish the scheme E from a random permutation (the

notations are the ones of Figure 1), one uses pairs (P_1, P_2) such that $P_1 \oplus P_2 = \Delta_i$ and evaluates the correlation ϵ of the linear approximation $\lambda_o \cdot E(P_1) = \lambda_o \cdot E(P_2)$. The computation of this correlation was first done based on two main assumptions. First observe that the linear equation $\lambda_o \cdot C_1 = \lambda_o \cdot C_2 \iff \lambda_o \cdot C_1 \oplus \lambda_o \cdot C_2 = 0$ can be rewritten as:

$$(\lambda_o \cdot C_1 \oplus \lambda_i \cdot Y_1) \bigoplus (\lambda_i \cdot Y_1 \oplus \lambda_i \cdot Y_2) \bigoplus (\lambda_i \cdot Y_2 \oplus \lambda_o \cdot C_2) = 0$$

By assuming the independence between the intermediate linear combinations, one can use Matsui's Piling-up lemma in [Mat94] to compute the correlation ϵ. The second assumption concerns the computation of the correlation of $\lambda_i \cdot Y_1 \oplus \lambda_i \cdot Y_2$ which is equal to $\lambda_i \cdot \Delta_o$ if the differential is respected. Then:

$$\begin{aligned}\mathbb{P}[\lambda_i \cdot (Y_1 \oplus Y_2) = 0] = \mathbb{P}(\lambda_i \cdot (Y_1 \oplus Y_2) = 0 \,|\, Y_1 \oplus Y_2 = \Delta_o) \cdot p \\ + \mathbb{P}(\lambda_i \cdot (Y_1 \oplus Y_2) = 0 \,|\, Y_1 \oplus Y_2 \neq \Delta_o) \cdot (1 - p)\end{aligned}$$

Assuming that $\lambda_i \cdot (Y_1 \oplus Y_2) = 0$ with probability $\frac{1}{2}$ when $Y_1 \oplus Y_2 \neq \Delta_o$, then we have:

$$\mathbb{P}[\lambda_i \cdot (Y_1 \oplus Y_2) = 0] = \frac{1}{2} \pm \frac{p}{2}.$$

So the correlation of $\lambda_i \cdot Y_1 \oplus \lambda_i \cdot Y_2$ is $\pm p$ and the correlation of the two other parts is q as they follow the linear characteristic. Then the correlation of the differential-linear characteristic is $\epsilon \approx \pm pq^2$.

2.2 Enhancing the Differential-Linear Cryptanalysis

The previous assumptions do not hold in practice, giving correlations far from experimental results. In fact, these hypotheses ignore completely the dependency between the differential and the linear part. Bar-On *et al.* in [BDKW19] showed the impact of that dependency on the DL attack complexity and they introduced the DLCT to overcome that issue and take advantage of the dependency in choosing the best characteristic that improves the attack complexity. For an $n \times n$ S-box S, the DLCT is defined as follows:

$$DLCT_S(\Delta, \lambda) := \#\{x \in \mathbb{F}_{2^n} \,:\, \lambda \cdot S(x) \oplus \lambda \cdot S(x \oplus \Delta) = 0\} - 2^{n-1}$$

The introduction of the DLCT came with a new framework for the DL attack that was introduced in [BDKW19]. That framework proposes to split the cipher E into three part E_0, E_m and E_1 (*cf.* Figure 2) such that $E = E_1 \circ E_m \circ E_0$ where the middle part E_m aims to handle the dependency between the two others. The correlation of the middle part is computed using the DLCT and then the whole correlation for the DL attack using this framework is approximated by $\epsilon = 4p\mathcal{R}q^2$, where the $\mathcal{R} = \frac{DLCT_S(\Delta_0, \lambda_i)}{2^n}$ is the bias of the middle part. However, this estimate is still a bit far from experimental results.

Hadipour, Derbez and Eichlseder proposed in [HDE24] a generalization of the DLCT framework giving a more precise approximation of the distinguisher

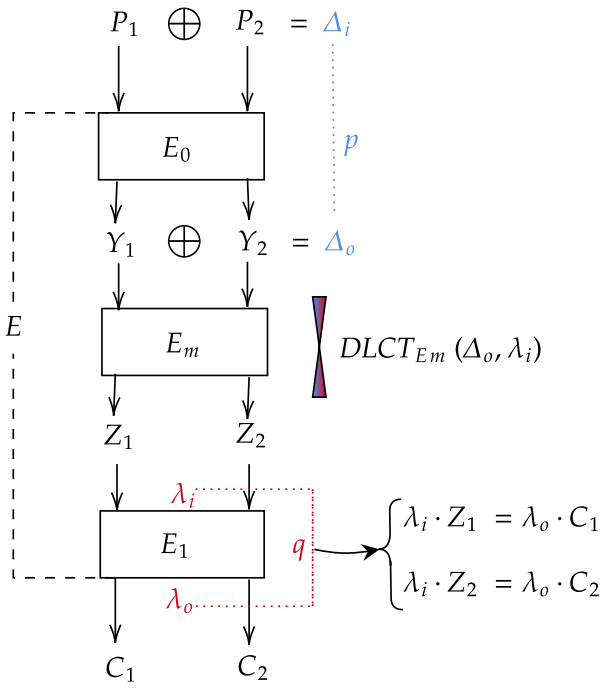


Fig. 2: Structure of a differential-linear distinguisher with middle part.

correlation: $\epsilon = prq^2$ where r is the correlation of the middle part. In their paper, they rewrote the DLCT definition for an S-box S as:

$$DLCT(\Delta_i, \lambda_o) = |DLCT_0(\Delta_i, \lambda_o)| - |DLCT_1(\Delta_i, \lambda_o)|$$

where $DLCT_b(\Delta_i, \lambda_o) = \{x \in \mathbb{F}_{2^n} : \lambda_o \cdot S(x) \oplus \lambda_o \cdot S(x \oplus \Delta_i) = b\}$. And, one can formalize the two main properties exploited to model the middle part differential/linear propagation in [HDE24] as follows:

$$\text{Cell-wise switches: } \forall \Delta_i, \lambda_o, DLCT(\Delta_i, 0) = DLCT(0, \lambda_o) = 2^n \tag{1}$$

$$\text{Bit-wise switches: } \exists \Delta_i \neq 0, \lambda_o \neq 0, DLCT(\Delta_i, \lambda_o) = \pm 2^n. \tag{2}$$

Moreover, instead of considering only the input difference and the output mask, they also take into account other parameters such as the output difference, the input mask, or intermediate differences and/or masks to introduce new variants of the DLCT, such as Upper/Lower/Extended/Double DLCT. Those tables permit to compute more precisely the correlation r.

In [HDE24], authors also introduced a new automatic tool for exploring differential-linear distinguishers. Their approach consists of decomposing the scheme E as before in three subciphers such that $E = E_1 \circ E_m \circ E_0$ and performing probabilistic differential propagation on E_0 and a probabilistic linear propagation on E_1. Specifically for the middle part, a deterministic differential propagation is performed in the forward direction and a deterministic linear propagation is performed in the backward direction in order to capture every cell/bit-wise switches according to the implemented model. In fact, this tool comes with a cell-wise model, considering an S-box as a black-box which is either active or not, for strong aligned schemes and a bit-wise model for weakly aligned schemes. Cell-wise and bit-wise switches refer to transitions with correlation ± 1 (see equations (2) and (1)). For the cell-wise approach they are characterized by a zero input differential or zero output mask and for the bit-wise model they are characterized by the deterministic transitions of the S-boxes. Therefore, maximizing these transitions may allow a higher correlation for the middle subcipher which now can be extended to more than one round thanks to the variants of the DLCT.

They applied this new tool to various ciphers leading to new distinguishers that slightly enhance the already existing distinguishers. This motivates a further usage of their tool as presented in this work.

2.3 Key Recovery in Differential-Linear Attack

Overall Complexity. In this subsection we give a slight overview of a differential-linear key recovery attack as described in [BCD+22]. Suppose we have a r_d-round distinguisher $Dist_{DL}$ of a scheme E with characteristic $\Delta_i \xrightarrow{prq^2} \lambda_o$ constructed using the method of [HDE24]. The first step involves extending the distinguisher with r_u rounds in the differential part at the top and r_l rounds in the linear one at the bottom as shown in Figure 3. These extensions activate some key bits, let's denote κ_u the number of active key bits in the differential extended rounds r_u and κ_l the number of active key bits in the linear side. These extensions could happen with certain probabilities denoted by p_u for the upper part (in the differential direction) and q_l for the lower part (the linear one). The key recovery algorithm is given in Algorithm 2 of Appendix B.

The number of pairs needed to run this attack is

$$N \approx O([p_u pr(qq_l)^2]^{-2}) \tag{3}$$

More precisely, from the model [BN16] we have:

$$N \approx \frac{\phi^{-1}(1-2^{-a-1})^2 - \phi^{-1}(1-Ps/2)^2}{(ELP - 2^{-n})\phi^{-1}(1-Ps/2)^2} \tag{4}$$

where a is the advantage, Ps the success probability and $ELP = (p_u pr(qq_l)^2)^{-2} = \epsilon^{-2}$ and ϕ is the normal law distribution.

The time complexity is given by $c_d(N + c_l)$ where c_d is the number of times we run the 'for' loop in line 3 of Algorithm 2, so $c_d = 2^{\kappa_u}$. Inside that loop, the

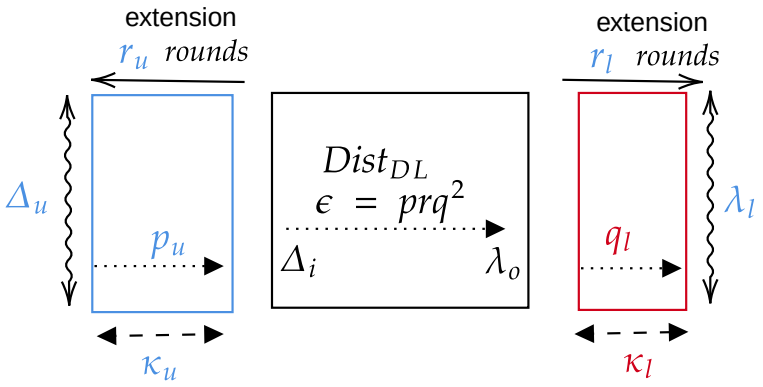


Fig. 3: Differential-linear key recovery

key recovery for the linear part is run. To efficiently perform this step, one can use the FFT algorithm introduced in [FN20] which has a cost of:

$$O(N) + O(2 \cdot (\rho_M + \rho_A \kappa_{out}) \cdot 2^{2\kappa_{out}+\kappa_{in}})$$

where $\kappa_{out} + \kappa_{in} = \kappa_l$ with κ_{out} the number of external active bits, κ_{in} the internal active bits, ρ_A the cost of n-bit addition and ρ_M cost of the n-bit product (n is the scheme block size). Therefore, the time complexity is:

$$\mathcal{T} \approx 2^{\kappa_u} \cdot (N + 2 \cdot (\rho_M + \rho_A \kappa_{out}) \cdot 2^{2\kappa_{out}+\kappa_{in}})). \tag{5}$$

Framework for an Optimal Key-Guessing. Broll *et al.* [BCFG+21] introduced a generic framework for optimizing key-bit guessing. Their approach consisted of transforming an S-box or some of its components into a binary decision tree and proved that the optimal number of guesses of the key bits was equivalent to the number of leaves of the corresponding optimal tree. This framework can drastically reduce the number of key bits needed to be guessed to determine an S-box output or some of its components. Therefore, we obtain a gain factor from the usual approach consisting in guessing all key bits. More precisely, let us consider that we are crossing an 4×4 S-box S_r in the upper extended rounds r_u. There are various possibilities according to its output differences and values. Let's represent the S-box as a column (see Figure 4), then we can identify various type of columns:

– *Column of type a: the S-box output difference is 0 and no output value is needed for the next round.* This situation corresponds to an inactive S-box, so no key guess is required.

– *Column of type b: the S-box has a zero output difference but one of its output value is needed.* In this case, one needs to guess the key in order to determine the required value $y_i, i \in 0, 1, 2, 3$. Indeed, the output value is given by the function

$$y_i(x + k) =< 2^i, S_r(x + k) >$$

which depends on the key bits entering the S-box. Therefore, we compute the optimal binary decision tree of this function. As shown in [BCFG+21], the number of leaves of this tree corresponds to the number of optimal key guesses for k.
– *Column of type c: the S-box has a zero output difference but two of its output value are needed.* Similar to the previous case, the only change is the function determining the output values which, in this case, is $y_{i,j}(x + k) =< 2^i + 2^j, S_r(x + k) >$ if the needed values are y_i and y_j.
– *Column of type d: variable output differences but no output value is needed.* In this case, for a possible output difference Δ, we need to ensure that it is satisfied. In other words, given x we need to determine x' such that :
$S_r(x + k) + S_r(x' + k) = \Delta \Leftrightarrow x' = S_r^{-1}(S_r(x + k) + \Delta) + k$
$\Leftrightarrow x + x' = S_r^{-1}(S_r(x + k) + \Delta) + x + k = F_\Delta(x + k)$.
Determining x' is equivalent to evaluating F_Δ at $x + k$. For a given Δ, the optimal number of key guesses corresponds to the number of leaves of the optimal tree of F_Δ. The overall number of key guesses is the average across values of Δ.
– *Column of type e: the S-box has a fixed output difference Δ.* For this type of column, we need to ensure the difference Δ is satisfied, i.e, the pair $(x, x+\delta)$ satisfies:
$S_r(x+k)+S_r(x+k+\delta) = \Delta \Leftrightarrow \delta = S_r^{-1}(S_r(x+k)+\Delta)+x+k = F_\Delta(x+k)$
$\Leftrightarrow x + k \in F_\Delta^{-1}(\delta)$.
Let $g_\Delta^\delta(x) : \mathbb{F}_2^k \to \mathbb{F}_2; g_\Delta^\delta(x) = 0 \Leftrightarrow x \in F_\Delta^{-1}(\delta)$. Then a pair $(x, x + \delta)$ is valid if, and only if, $g_\delta^\Delta(x+k) = 0$. Therefore, verifying the validity of a pair is equivalent to evaluation g_Δ^δ at $x + k$. The number of guesses for this type of column is the average taken over all the possible values of δ.
– *Column of type f: the S-box has a variable output difference and 1 to 3 output values are needed.* According to the output difference, these columns are of type b/c (difference is zero) or of type e (difference is fixed).

The optimal trees can be computed thanks to the tool provided in [BCFG+21].

Gain factor. This representation of an S-box makes it possible to perform early filtering with an optimal number of key guesses for pairs crossing the extended rounds, yielding a gain factor defined as follows:

$$\frac{\#\text{The optimal number of guesses}}{\#\text{The maximum number of guesses}}$$

where the optimal number of guesses is given by the number of leaves of the optimal tree, and the maximum number is guesses is 2^κ with κ is the bit length of the key entering the S-box (4 bit here).

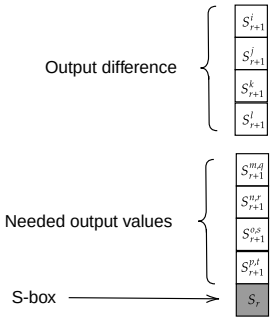


Fig. 4: Output differences and needed values of the S-box S_r of the round r in the upper extended rounds r_u. In the output difference, a cell containing S^i_{r+1} is influenced by the input difference of the i-th S-box of the next round. And in the values, a cell containing $S^{m,q}_{r+1}$ means its output value is needed to determine the q-th bit of the m-th S-box in the next round.

Key absorption. Some key bits of the next round can be absorbed. In fact, let y_0 be a needed output value of S_r. The tree giving the value of y_0 yield a linear relation of y_0 depending on the key k_r, i.e $y_0 =< \alpha, k_r > +\beta$. Recall, y_0 influences the q-th bit of the input of the m-th S-box in the next round. Therefore, that q-th bit involves the linear combination $y_0 + k^q_{r+1} =< \alpha, k_r > +\beta + k^q_{r+1}$. So instead of determining y_0, one can determine the linear combination $y_0 + k^q_{r+1}$ and so absorb the key bit k^q_{r+1} of the next round.
This gives a gain factor of 2^{-1}.

Example. See figure 5. In our example, we are using the SERPENT S-box S_0. This example is a column of type b and we need the output value y_0 as it influences the input Δx_3 of the sixteenth S-box of the next round. The optimal tree for $y_0(x)$ is drawn in Figure 12 of Appendix D and has 6 leaves. So we only need 6 guesses in average instead of 16 to get y_0. In fact, y_0 is obtained using the algorithm below, which is derived by traversing the optimal tree.

if $x_2 \oplus x_1 \oplus x_0 = 0$ **then** ▷ Guess $k_2 \oplus k_1 \oplus k_0$
  if $x_2 = 0$ **then** ▷ Guess k_2
    $y_0 = x_3 \oplus 1$ ▷ Guess k_3
  else
    $y_0 = x_1 \oplus 1$ ▷ Guess k_1
  end if
else
  $y_0 = x_3 \oplus x_1 \oplus 1$ ▷ Guess $k_3 \oplus k_1$
end if

So the gain factor is 6/16. Each possible guess provides a linear combination for y_0, allowing us to absorb the key bit into the input Δx_3 of the sixteenth S-box

in the next round. This yields a total gain factor of $2^{-1} \cdot 6/16$. There are other examples in subsection 4.1.

Fig. 5: Column Example.

Overall Complexity. Applying this framework in the case of the DL key recovery, one can have a gain 2^{g_u} for the guessing of the κ_u bits in the upper extension giving the time complexity of:

$$\mathcal{T} \approx 2^{\kappa_u + g_u} \cdot (N + 2 \cdot (\rho_M + \rho_A \kappa_{out}) \cdot 2^{2\kappa_{out} + \kappa_{in}}). \tag{6}$$

3 Our Approach

As finding the best distinguisher does not always ensure to find the best attack, we decide to integrate the search for a DL attack inside the code of the DL distinguishers. Therefore, our tool searches at the same time a DL distinguisher and the corresponding extended rounds for key recovery for the whole DL attack. We start by studying the DL distinguishers provided in [HDE24] and the way they work. Then, we see how to add the rounds of the key recovery to those DL distinguishers. Inspired by Broll *et al.* [BCD+22], we tuned the model so it can fix some transitions in the extended rounds and even set some bits in the truncated differences.

3.1 Finding DL Distinguishers

As previously described, the permutation E for the DL distinguisher is divided as $E = E_1 \circ E_m \circ E_0$. E_0 has r_0 rounds and corresponds to the differential part, E_m has r_m rounds and corresponds to the rounds where both the influence of the differential and the linear parts are studied and E_1 has r_1 rounds and corresponds to the search of the linear trail.

The proposed automatic tool by Hadipour *et al.* [HDE24] is a CP (Constraint Programming) model. More precisely, it is the chaining of four CSP (Constraint Satisfaction Problem) models: the first model CSP_0 searches for a differential trail in E_0 with probability $p = 2^{-\mathfrak{p}}$; the second CSP_{mu} and third CSP_{ml} search for a deterministic differential and a deterministic linear propagation in the middle rounds E_m respectively and the fourth one CSP_1 looks at a linear trail on E_1 with correlation $q = 2^{-\mathfrak{q}}$. Then, the objective function obj_1 is thus to minimize:

$$obj_1 = \min(w_u \cdot \mathfrak{p} + w_m \cdot cm + w_l \cdot 2\mathfrak{q})$$

where:

- w_u is the weight of the differential part and is proportional to the differential uniformity ($\mathcal{DU}$) of the corresponding S-box;
- w_l is the weight of the linear part and is proportional to the square of the linearity ($\mathcal{L}$) of the corresponding S-box;
- w_m is the weight of the middle part and is proportional to the differential linear uniformity($\mathcal{DLU}$) of the corresponding S-box;
- cm is the number of common activated bits in the deterministic differential propagation and the linear one in both directions. It represents the intersection of the activated bits both in the differential side and in the linear side in E_m where both influences are studied.

For example in the case of SERPENT, we have $w_u = 2, w_l = 2, w_m = 1$. For the PRESENT case, we have $w_u = 1, w_l = 1, w_m = 2$.

The differential part (resp. linear part) is fully determined by its model CSP_0 (resp. CSP_1), i.e., we have its input/output differences (resp. masks) together with the corresponding probability (resp. correlation). However, for the middle part E_m, the two models (CSP_{mu} and CSP_{ml}) are deterministic, i.e., the differential and linear trails in the middle propagate with probability 1, and the objective is to capture the maximum number of bit-switches (see Equation (2)) as shown in [HDE24]. Consequently, we cannot derive the correlation r from these models, as it is an intrinsic probabilistic property of the middle part. Therefore, r is determined experimentally or by using the generalized DLCT framework when cm is enough small.

We noticed that minimizing the common active bits cm of the middle part E_m does not guarantee the best correlation r of the middle part E_m. In fact, we have observed that some middle parts with a slightly worse cm exhibit a much higher correlation for the r_m middle rounds (see Table 2). In Table 2, the CP model, guided by the objective function, would prioritize outputs with lower cm. However, we can gain a factor of 2^5 (resp. 2^2) in the correlation of the middle part for SERPENT (resp. PRESENT) at the cost of a slightly worse cm. Therefore, we added a way to explore some other options for the middle part from the output of the CP model. The idea is to find a middle part with a higher correlation such that it can be extended to a distinguisher whose obj_1 function is not that far from the output of the CP model. This approach will be detailed in subsection 3.3.

Table 2: Comparing middle part correlation r and their common active bits cm. $\mathfrak{p}$ and $\mathfrak{q}$ corresponds respectively to the $-log_2(P)$ and $-log_2(C^2)$ with P the probability of the best r_0 rounds differential extension and with C the correlation of the best r_1 rounds linear extension of this middle part characteristic. The offset represents the starting S-box for the middle part. The upper characteristics correspond to the SERPENT case with 3 rounds in the middle part where the lower to PRESENT with 8 rounds in the middle part.

Input difference Δ_m	output mask λ_m	offset	cm	r	$\mathfrak{p}$ $2\mathfrak{q}$
00000010040000004000000000000208	00000080000000000000008000001000	4	32	$2^{-12.06}$	7 40
00000010040000004000000000000208	20000000000000002000000000000004	4	39	$2^{-11.05}$	7 40
00000010040000004000000000000208	00000008000000000000000800000100	4	35	$2^{-10.3}$	7 40
00000008020000002000000000000104	00008000000000000000800000100000	4	38	$2^{-7.37}$	7 40
04000000000300000010000082000000	00000004000000000000000400000080	4	41	$2^{-12.55}$	7 40
04000000000300000010000082000000	00000001000000000000000100000020	4	43	$2^{-13.19}$	7 40
04000000000300000010000082000000	02000000000000000200000040000000	4	43	$2^{-9.84}$	7 40
0010000000000c00000040000208000	00400000000000000040000008000000	4	43	$2^{-7.42}$	7 40
$0x0000009000000000$	$0x0000000000200000$	-	212	$2^{-7.7}$	4 16
$0x0000000000000900$	$0x0000200000000000$	-	212	$2^{-8.9}$	4 16
$0x0000000090000000$	$0x0400000000000000$	-	212	$2^{-9.9}$	4 16
$0x0000000000000009$	$0x0020000000000000$	-	212	$2^{-8.42}$	4 16

3.2 Finding Attacks

The obj_2 Function. To turn DL distinguishers into DL attacks, we directly implement the attack described in Subsection 2.3 as an extension of Hadipour's distinguishers. As previously mentioned, we extend E by r_u rounds at the beginning with a CSP_u model and r_l rounds at the end with a CSP_l model, denoting by κ_u and κ_l the key bits to guess in those two extensions. We also denote by $p_u = 2^{-\mathfrak{p}_u}$ the probability of the extension in the differential part and $q_l = 2^{-\mathfrak{q}_l}$ the correlation of the linear part extension. Indeed, the two ciphers studied are bit-oriented ciphers and thus, the backward and forward extensions could be probabilistic. So, we first need to tune the objective function obj_1 into obj_2 according to those probabilities. So obj_2 empirically becomes:

$$obj_2 = \min(obj_1 + \mathfrak{p}_u + 2\mathfrak{q}_l + SB_u + SB_l)$$

where SB_u is the number of active S-boxes in the differential side and SB_l in the linear side. Since the number of key bits to guess is proportional to the number of active S-boxes in SPN-like ciphers, the term SB_u+SB_l in obj_2 serves to minimize the number of active S-boxes in the extended rounds and, consequently, the number of key bits to be guessed. Note also that the extended rounds are partially deterministic (see next subsection); therefore, minimizing the probability weight alone is not sufficient to reduce the number of active S-boxes.

One can notice that the distinguisher constraints are not altered, as obj_1 is kept unchanged and the added terms are weighted with coefficient 1. This is because we do not want to distort obj_1 significantly. Thus, the model can

relax the strict minimization of the distinguisher in order to find better attacks. The goal is to obtain distinguishers with slightly lower correlation than those proposed in [HDE24], but more effective for the key recovery phase.

Probabilistic extensions. Very often in the literature, the extensions of distinguishers are deterministic. That is, the S-boxes in the extended rounds are traversed with certainty, meaning their inputs or outputs are truncated and not known. However, this approach quickly increases the number of key bits that must be guessed and becomes impractical as the number of extended rounds grows, especially for schemes with strong diffusion properties. To overcome this issue and reduce the number of key bits to guess, an alternative is the probabilistic extension. This idea was first introduced in [BCD+22] for differential-linear attacks, where the authors fixed certain S-boxes input differences in the extended rounds. These S-boxes then become part of the overall distinguisher with probability p_u and contribute to its probability, thereby increasing the data complexity. Despite this drawback, the probabilistic extension offers two main advantages. First, the fixed S-boxes are integrated into the distinguisher, so their associated key bits no longer need to be guessed, reducing the overall key-guessing complexity. Second, these extensions introduce early filters that allow to eliminate invalid pairs at an earlier stage.

To automate this approach, we model the S-boxes in the extended rounds so that they can adopt either deterministic or probabilistic behavior. This is achieved by combining the deterministic and probabilistic models using an OR operation. An S-box is set to be deterministically crossed if its input difference (resp. output mask) is unknown in the upper (resp. lower) extension; otherwise, it is crossed probabilistically.

For example, suppose that the inputs/outputs of a 4×4 S-box S are denoted by $(b_i)_{i=0..3}$ and $(a_i)_{i=0..3}$, where each $a_i, b_i \in \{-1, 0, 1\}$. Here, 0 and 1 represent the actual bit values of a_i and b_i, while -1 indicates that the bit is unknown. An input or output is said to be unknown if at least one of its components is equal to -1. If this S-box is in the upper extended rounds, we are modeling the inverse of S, S^{-1}, $((b_i)_{i=0..3} \xleftarrow{p} (a_i)_{i=0..3})$, then:

- A deterministic transition can be, for instance, $(-1,-1,-1,-1) \xleftarrow{p=1} (0,1,1,1)$, which corresponds to the trivial case. Note that the LSB is the most left component.
- A non-trivial deterministic transition occurs when at least one input bit is known. For example, $(-1,1,-1,-1) \xleftarrow{p=1} (0,1,0,0)$.
- A probabilistic transition, on the other hand, could be $(1,0,0,0) \xleftarrow{p=1/4} (0,1,1,1)$ or $(1,0,0,1) \xleftarrow{p=1/8} (0,1,1,1)$.

To model the deterministic transitions of the S-box, we first determine all its non-trivial deterministic transitions which can be derived from the DDT. The deterministic model is the following one (with the inverse of the S-box S): *Given an input $a = (a_i)_{i=0..3}$ if there is an non-trivial output $b = (b_i)_{i=0..3}$ then return*

b otherwise return $(-1,-1,-1,-1)$.

The probabilistic model corresponds to the standard representation of an S-box in a distinguisher, i.e., the modeling of its DDT in this case. Both the deterministic and probabilistic models can be directly obtained using the open-source S-box analyzer tool described in [HNE22].

Concretely, say S is the inverse of the SERPENT s-box S_0. All its non-trivial deterministic transitions are: $(-1,1,-1,-1) \xleftarrow{p=1} (0,1,0,0)$, $(-1,1,-1,-1) \xleftarrow{p=1} (1,0,0,0)$ *and* $(-1,0,-1,-1) \xrightarrow{p=1} (1,1,0,0)$. And let sbox_diff represent its DDT modeling. Then algorithm 1 given in Appendix B, represent the combination of its deterministic model and its probabilistic one.

Finally, in the case of a trivial deterministic transition, we added the option for the model to force at least one of the b_i values to zero. This can increase the number of inactive S-boxes in the previous round, thereby reducing the number of key bits that need to be guessed. Indeed, the output difference of the previous round $(r-1)$ can be computed from the input difference of the current round (r) via the inverse of the linear transformation. Consequently, if there are few active S-boxes in round r, some S-boxes in round $r-1$ will have output differences that depend on only a single input bit difference from round r. If this bit is set to zero, the corresponding S-box output difference in round $r-1$ becomes null. We implemented this idea through the predicate fixed_transition, which takes as input a trivial deterministic transition and forces at least one of the truncated bit differences to 0. For example, it can change $(-1,-1,-1,-1) \xleftarrow{p=1} (0,1,0,0)$ into $(0,-1,-1,-1) \xleftarrow{0<p<1} (0,1,0,0)$, where p is computed from the DDT (our modeling ensure that at leat one of such a transition exists ($p \neq 0$) in the DDT). These transitions act as additional filters that are crossed with a certain probability, thereby increasing the data complexity.

Similar modeling can be adopted for the S-boxes in the linear extension using the LAT instead of the DDT and the S-boxes are crossed forward (in contrast to the upper extension where we model the inverse of the S-boxes). One can then capture the correlation q_l of the fixed transitions that will be integrated to the distinguisher. However, in the linear side fixing many transition might be very expensive as the correlation is squared in the overall distinguisher correlation.

3.3 Tuning the DL Distinguishers

We added a Python module that takes as input our CP model and estimates the time and data complexities. When the output does not yield a viable attack, our tool refines the middle part by exploring all rotations of the middle-part characteristics. This approach is motivated by the fact that some substantially equivalent CP solutions may provide a better correlation but with a slightly worse cm, and therefore a worse obj_1. Two outputs of the CP model, s_1 and s_2, are considered substantially equivalent if:

$$|(obj_1(s_1) - w_m \cdot cm(s_1)) - (obj_1(s_2) - w_m \cdot cm(s_2))| = |w_u \cdot (\mathfrak{p}_1 - \mathfrak{p}_2) + w_l \cdot (\mathfrak{q}_1 - \mathfrak{q}_2)| \leq \eta$$

with $\eta = \epsilon \cdot (w_u + w_l)$ for a small ϵ, e.g $\epsilon \leq 2$. In other words, the differentials probabilities p_1, p_2 do not differ two much and the same thing happens for the linear trails correlations q_1 and q_2.
Table 2 contains substantially equivalent solution characteristics, as we can observe that for given solutions s_1 and s_2, we have $p_1 = p_2$ and $q_1 = q_2$. This table is constructed using the rotational approach, which we define as follows: given a middle-part characteristic $\Delta_m \rightarrow \Lambda_m$, the other options obtained through rotations are given by the set:

$$\mathcal{R} = \{\pi_1(\Delta_m) \rightarrow \pi_2(\Lambda_m) : \pi_1 \, and \, \pi_2 \text{ are two bitwise cyclic rotations over the full state}\}.$$

This approach allows us to reach some solutions that are substantially equivalent to $\Delta_m \rightarrow \Lambda_m$. It is an ad hoc method that we integrated into our external Python module. However, this set does not guarantee a better middle part than $\Delta_m \rightarrow \Lambda_m$. Since the set $\mathcal{R}$ is finite, there exists a maximum achievable correlation within it, and this maximum may occur for $(\pi_1, \pi_2) = (id, id)$. This method simply provides a way to go beyond the initial CP model, either by compensating for limited running time or by reconsidering solutions that were previously discarded due to a worse obj_1 value.

Our tool computes the experimental correlation for each of these options. It will, at the same time, identify among these options those that are substantially equivalent to the MiniZinc output but with a better correlation than the initial middle part. The complexity of this approach is roughly $4n^2r^{-2}$ encryptions, where r is the correlation of $\Delta_m \rightarrow \Lambda_m$ and n is the cipher block size. Indeed, there are n^2 possible rotations (π_1, π_2), and for each of them we must compute the correlation of $\pi_1(\Delta_m) \rightarrow \pi_2(\Lambda_m)$. Since we are only searching for middle parts with correlation higher than r, it suffices to test at most $4r^{-2}$ pairs.

We also provide a module to extract a full distinguisher from a middle part or to check a distinguisher given its characteristics. Therefore, in the case of a promising option for the middle, one can extend it to get the corresponding distinguisher for the attack.

4 Attacks

We applied our approach to SERPENT and PRESENT. For SERPENT with a 256-bit key, the best known attacks in the literature include linear [BDK02a, CSQ07] and multidimensional linear attacks [HCN08, CSQ08, NWW11], nonlinear attacks [MC13], boomerang and rectangle attacks [KKS01, BDK02b, BDK01], and differential–linear attacks [BDK03, DIK08, LLL21, BCD$^+$22, HDE24]. As a result of our work, we provide, to the best of our knowledge, the best differential–linear key-recovery attacks (time/data) on 12 rounds of SERPENT.

Concerning key recovery attacks on PRESENT, differential cryptanalysis reaches 18 rounds [BDD$^+$24] for PRESENT-80 and the best attacks are linear ones reaching 28 rounds [FN20] and even 29 rounds in [WLW24]. Thus, we will introduce the first differential-linear key recovery attacks of PRESENT. They reaches 16 rounds for PRESENT-80 and 18 rounds for PRESENT-128.

4.1 SERPENT

SERPENT, designed by Anderson, Biham and Knudsen [BAK98], is a bit-oriented block cipher, finalist of the AES competition.

Description of SERPENT SERPENT internal state X is 128-bit long and its key size is 128, 192 or 256 bits. SERPENT has 32 rounds. The internal state is represented by four words X_0, X_1, X_2, and X_3 of 32 bits each such that $X = X_3||X_2||X_1||X_0$. The round function of SERPENT consists of a key addition followed by a parallel application of 32 S-boxes followed by a linear transformation. The cipher uses 8 different 4×4 S-boxes (see Table 3) and each round uses one S-box in parallel. A round is thus composed of the following operations:

Table 3: The 8 SERPENT S-boxes.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
S0	3	8	F	1	A	6	5	B	E	D	4	2	7	0	9	C
S1	F	C	2	7	9	0	5	A	1	B	E	8	6	D	3	4
S2	8	6	7	9	3	C	A	F	D	1	E	4	0	B	5	2
S3	0	F	B	8	C	9	6	3	D	1	2	4	A	7	5	E
S4	1	F	8	3	C	0	B	6	2	5	4	A	9	E	7	D
S5	F	5	2	B	4	A	9	C	0	3	E	8	D	6	7	1
S6	7	2	C	5	8	4	6	B	E	9	1	F	D	3	A	0
S7	1	D	F	0	E	8	2	B	7	4	C	A	9	3	5	6

- **Key addition (`KeyAdd`):** The round key K_i for round i is XORed to the internal state. Those round keys are obtained from the master key K using the SERPENT key schedule algorithm.
- **S-box layer ($\texttt{SL}_i$):** The S-box used in round i is $S_{i \bmod 8}$ and is applied in parallel to words $(X_3[j], X_2[j], X_1[j], X_0[j])_{j=0..31}$ where $X_w[j]$ corresponds to the j-th bit of the word X_w where $X_w[0]$ is the Least Significant Bit.

– **Linear transformation (LL):** The SERPENT linear transformation on $X = X_3||X_2||X_1||X_0$ is defined as follows:

$$
\begin{aligned}
X_0 &= X_0 \lll 13 \\
X_2 &= X_2 \lll 3 \\
X_1 &= X_1 \oplus X_0 \oplus X_2 \\
X_3 &= X_3 \oplus X_2 \oplus (X_0 \ll 3) \\
X_1 &= X_1 \lll 1 \\
X_3 &= X_3 \lll 7 \\
X_0 &= X_0 \oplus X_1 \oplus X_3 \\
X_2 &= X_2 \oplus X_3 \oplus (X_1 \ll 7) \\
X_0 &= X_0 \lll 5 \\
X_2 &= X_2 \lll 22
\end{aligned}
$$

Given the state X_i, the state after round i is computed as $X_{i+1} = \mathtt{LL}(\mathtt{SL}_i(X_i \oplus K_i)$ for i from 0 to 31.

The Attack on 12 Rounds. The best known key recovery attack on SERPENT, to the best of our knowledge, is the differential-linear attack which is presented in [BCD+22] in the single-key setting and covers 12 rounds. This attack improves earlier ones by using a conditional key-recovery approach based on an old 9-round distinguisher of SERPENT. This distinguisher and its extended rounds are shown in Figure 10 of Appendix A. It consists of 3 differential rounds (#4 to #7) with probability 2^{-6}, followed directly by 6 linear rounds (#10 to #17) with correlation 2^{-26} (after removing the truncated difference in #17), yielding an overall correlation of 2^{-58}. Until the aforementioned work, this distinguisher provided the best basis for key-recovery attacks on SERPENT.

Hadipour et al. [HDE24] proposed distinguishers with significantly higher correlations than the old distinguisher; however, their extensions do not yield a 12-round attack on SERPENT. The efficiency of their tool in finding high-correlation distinguishers motivated this work. Our objective was to identify distinguishers sufficiently close to those in [HDE24], but that could also support viable attacks over at least 12 rounds of SERPENT. A comparison between their distinguishers and ours is provided in Table 4. As a result, our approach yields a distinguisher with $r_0 = 2$, $r_m = 3$, and $r_1 = 4$, presented in Table 5 and illustrated in Appendix E. The correlation of this distinguisher is $\epsilon = prq^2 = 2^{-55.38}$.

This distinguisher is obtained thanks to the rotational approach described in Subsection 3.3. Indeed, the CP program gave a substantially equivalent distinguisher ($p = 2^{-8}$ and $q^2 = 2^{-40}$) but with a middle part correlation smaller than 2^{-12}. Therefore, giving an overall correlation smaller that 2^{-60}, this distinguisher could not help to improve the attacks on SERPENT. However, exploring other options given by the rotations, we found the distinguisher of table 5 with a

Table 4: Comparison between the distinguishers from [HDE24] and ours. Although theirs achieve higher correlation, ours provide a better compromise for key-recovery attacks and thus enable successful attacks over a larger number of rounds.

Tool	Scheme	Dist. nb rounds	Dist. correlation	Key rec. nb rounds
[HDE24]	SERPENT	9	$2^{-54.43}$	< 12
	SERPENT	9	$2^{-50.95}$	12^a
	PRESENT-(80/128)	13	$2^{-23.77}$	< 16
Ours	SERPENT	9	$2^{-55.38}$	12
	PRESENT-80	13	$2^{-27.72}$	16
	PRESENT-128	13	$2^{-27.72}$	18

[a] Almost full codebook. Does not even perform as well as the attack in [BCD+22].

Table 5: New DL distinguisher characteristics for 9-rounds of SERPENT.

offset=2, $r_0 = 2, r_m = 3, r_1 = 4$	$p = 2^{-8}, r = 2^{-7.38}(exp.), q^2 = 2^{-40}$
Δ_i : 00000000000000480000000000000008	Δ_m : 00000080200000020000000000000104
Λ_m : 00008000000000000000800000100000	Λ_o : 00840108000000080084214810842100

much higher middle part correlation and leading to possibly better key recovery attacks.

The extension of our distinguisher is shown in Figure 6. It is extended by two rounds (#1–#3) at the top and by one round (#18) at the bottom. However, we treat the last round of the distinguisher (#16) as a part of the lower extension, making one of its transitions deterministic and thereby activating additional S-boxes in the lower part.

Upper extension: The S_0 S-boxes of round #1 are represented as columns in Figure 7. Since key guesses are unnecessary only for columns of type a, and there is only one such column in round #1, we must guess the key bits for the remaining 31 S-boxes, i.e., 124 bits of k_0. In the next round, two transitions are fixed ($S_1(0xd) = 0x4$ and $S_1(0xc) = 0x1$) and thus contribute to the distinguisher, while the other three transitions ($S_1(-1,-1,-1,0) = 0xe$, $S_1(-1,0,-1,-1) = 0xa$, and $S_1(0,-1,-1,-1) = 0x2$) are truncated. For this round, we must guess the key bits associated with the truncated transitions, i.e., 12 bits of k_1. Therefore, in the upper extension, a total of 136 key bits must be guessed.

Lower extension: This involves round #16, in which one transition is truncated, requiring 4 bits of k_{10} to be guessed, and round #18 with 13 active S-boxes, requiring 52 bits of k_{11}. Thus, the total number of key bits in the lower extension is 56.

The sieving steps

Upper extension: The sieving algorithm given in Algorithm 3 of Appendix B describes the construction of pairs with respect to the filters. This sieving process has a cost of 2^{-4}. More precisely, due to their fixed bits, each truncated transition has a probability of satisfaction as follows:

- Transition $(0,-1,-1,-1) \xrightarrow{S_1}_{DDT} (0,1,0,0)$ holds with probability $\frac{1}{4}$.
- Transition $(-1,0,-1,-1) \xrightarrow{S_1}_{DDT} (0,1,0,1)$ holds with probability $\frac{1}{2}$.
- Transition $(-1,-1,-1,0) \xrightarrow{S_1}_{DDT} (0,1,1,1)$ holds with probability $\frac{1}{2}$.

These probability are computed using the DDT of S_1.
The probability of the sieving process is 2^{-4}. Therefore, if N denotes the number of pairs entering the distinguisher, the data complexity is $D = 2^4 \cdot N$.
Lower extension: The same analysis, as done in [BCD+22], which we recall below, holds.
In round #16 (S-box S_2), we impose the condition that y_2 and y_0 must not both be equal to 1 simultaneously for the linear approximation $\langle B, x\rangle \oplus \langle 1, S_2(x)\rangle$. Without this condition, the correlation of this approximation is $\frac{4-12}{16} = -\frac{1}{2}$. Imposing this condition changes the correlation to $\frac{2-10}{12} = -\frac{2}{3}$ (see Table 6). This condition increases the overall correlation by a factor of $\left((2/3)\cdot(1/2)^{-1}\right)^4 = 2^{1.66}$. However we must discard ciphertexts that do not satisfy the condition, $(3/4)^2 = 2^{-0.83}$ of the ciphertexts remain, which reduces the data complexity by a factor of $2^{0.83-1.66} = 2^{-0.83}$, i.e $D = 2^{4-0.83} \cdot N = 2^{3.17} \cdot N$.

These conditions also have the effect of activating one more S-box in round #18, i.e., 56 bits of k_{11} must be guessed.

Table 6: Linear approximation: $\langle B, x\rangle \oplus \langle 1, S_2(x)\rangle$

	$(y_2, y_0) \neq (1,1)$													$y_2 = y_0 = 1$			
x	0	1	3	4	5	6	9	A	B	C	D	F		2	7	8	E
$S_2(x)$	8	6	9	3	C	A	1	E	4	0	B	2		7	F	D	5
$\langle B, x\rangle \oplus \langle 1, S_2(x)\rangle$	0	1	1	1	1	1	0	1	1	1	1	1		0	1	0	1

Data analysis

The overall distinguisher correlation is computed from the line 3 to the line 16 from Figure 6 and takes into account the sieving step of the previous subsection. The overall correlation is then: $\epsilon = 2^{-4-8-7.38-38+1.66} = 2^{-55.72}$.

Using the model of [BN16], the same than in [BCD+22], this attack needs $N \approx 2^{115.5}$ pairs, without sieving with an advantage of 38 guessed key bits with the same probability of success than in [BCD+22] *i.e.* 0.1. So a data complexity of $D \approx 2^{118.67}$ before sieving.

With this new distinguisher the probability of success can go up to 0.85 with an advantage of 38 bits and a worse number of pairs $N \approx 2^{121.84}$ (in total $D \approx 2^{125.01}$ for the data complexity).

Gain factor of the key guessing

We use the optimal key guessing framework describe in Section 2.3 to reduce the key guess cost in the upper extension. To do so, we represent the s-boxes in the s-box layer #1 as columns in Figure 7. And for each column, we determine the gain factor as follow:

Column of type a: there is only one column of this type so the gain factor is 2^{-4}.

Column of type b: We have six such columns and optimal trees for these columns are given in Figure 12 of Appendix D.

- For columns 31, 25, 22 and 2 we need to determine y_0. For each column the gain is $\frac{6}{16}$ and we can absorb $\Delta x_1, \Delta x_3$ of column 13 and Δx_3 of column 16. So the total gain is $2^{-3} \cdot (\frac{6}{16})^4 = 2^{-8.66}$
- In column 10, we need the value y_1. The gain factor of the corresponding tree is $\frac{6}{16}$ and we can absorb Δx_0 of column 16. So the gain is $2^{-2.415}$
- Finally column 21 determines y_2. We can absorb Δx_0 of column 30 and the number of tree leaves is 6. So the gain is $2^{-2.415}$

The total gain of these columns is then $2^{-13.49}$.

Column of type c: The corresponding optimal trees are given in Figure 13 of Appendix D.

- Optimal trees for determining (y_2, y_3) and (y_1, y_3) have both 8 leaves so a gain factor of $\frac{1}{2}$, the concerned columns are 1, 6 and 18. They allow to absorb the bit Δx_2 of column 16. So the total gain of these columns is $2^{-1} \cdot \frac{1}{2^3} = 2^{-4}$
- The optimal tree for determining (y_1, y_2) has 10 leaves so a gain factor of $2^{-0.68}$ for column 15.

The total gain for columns of type c is $2^{-4.68}$.

Column of type d: (see Figure 14 of Appendix D).
Recall that for these types of columns, no output value is required; we only need to ensure the output difference Δ of the S-box. The value of Δ is variable, and its possible values can be derived from the input differences of the next round (see Figure 7). Therefore, we can partially guess k_0 and determine the necessary input differences of the next round to compute the values of Δ. Once Δ is determined, we compute the optimal key guessing using the optimal tree of F_Δ.

We also observed that, depending on the conditions in the next round, some values of Δ are never taken for a valid pair. We detail this case for column 8.

- For column 8, we have $\Delta \in \{0, 2, 8, A\}$ with a probability of $\frac{1}{8}$. This probability is computed based on the bits of the relevant columns in #2, using

the DDT of S_1. However, upon closer inspection of Δ and considering the conditions in the next round, we observe that certain values of Δ are never possible for a valid pair (x, x'). Specifically, Δ can only be equal to 0. The probability is then $\frac{1}{8}$ for $\Delta = 0$ and 0 for all other values. Indeed, when $\Delta \in \{2, 8, A\}$, the resulting input difference in the next round is such that either Δx_0 of column 13 is 0 or Δx_2 of column 30 is 0. The possible input differences for these columns would be $\{0, 2, 4, 6\}$ for column 13 and $\{0, 2, 8, A\}$ for column 30. However, for both columns, none of these input values lead to the expected output differences in #3, since $DDT_{S_1}[\delta_{13}][14] = 0$ for all $\delta_{13} \in \{0, 2, 4, 6\}$ and $DDT_{S_1}[\delta_{30}][2] = 0$ for all $\delta_{30} \in \{0, 2, 8, A\}$.
Therefore, if $\Delta \in \{2, 8, A\}$, we can immediately discard the pair without guessing the remaining subkeys. As a result, the average gain factor is:

$$\frac{\frac{1}{8} \cdot 1 + 0 \cdot 12 + 0 \cdot 10 + 0 \cdot 8}{16} = 2^{-7}$$

– In column 14, $\Delta \in \{4, C\}$ with probability $\frac{1}{2}$, distributed as $\{\mathbb{P}(\Delta = 4) = 0, \mathbb{P}(\Delta = C) = 1/2\}$, with the corresponding number of leaves being $\{6, 10\}$. The gain in average:

$$\frac{0 \cdot 6 + \frac{1}{2} \cdot 10}{16} = 2^{-1.68}$$

– And in column 17, $\Delta \in \{8, 9, C, D\}$ with probability $\frac{1}{32}$, distributed as $\{\mathbb{P}(\Delta = 8) = 1/64, \mathbb{P}(\Delta = 9) = 1/64, \mathbb{P}(\Delta = C) = 0, \mathbb{P}(\Delta = D) = 0\}$, and a number of leaves equal to $\{10, 8, 10, 8\}$. So the gain factor is in average:

$$\frac{(\frac{1}{64} \cdot 10 + \frac{1}{64} \cdot 8 + 0 \cdot 10 + 0 \cdot 8)}{16} = 2^{-5.83}$$

The total gain factor is $2^{-14.51}$.

Column of type e: we have two columns with fixed output difference Δ. For these columns, we introduce the following function $y^{\delta}_{\Delta}(x)$ defined as follows: $y^{\delta}_{\Delta}(x) = (1 - g^{\delta}_{\Delta}(x)) \cdot < \alpha, S_0(x + \delta + k) + \Delta >$ where α is a mask indexing the needed values. This way we compute at the same time the corresponding trees of the needed values. Then, if $(x, x + \delta)$ is a valid pair i.e $g^{\delta}_{\Delta}(x) = 0$, we obtain the equation of the needed values $< \alpha, S_0(x + \delta + k) + \Delta >$.

Our approach consists of determining the possible input differences δ given the fixed output difference Δ and for each δ determining the optimal tree for y^{δ}_{Δ}.

– For column 4, $\Delta = 1$. Then the possible values for δ are : 3, 6, 9, C, B, A, E or F. From the DDT of S_0 we can see that these values have the same probability which is equal to $\frac{1}{8}$. The trees to computes are y^{δ}_1 for every δ with $\alpha = 12 = (0011)_2$. The gain factor is:

$$\frac{\frac{1}{8}(4 + 4 + 4 + 4 + 16 + 4 + 16 + 4)}{16} = 2^{-1.19}$$

– And for column 24, $\Delta = 5$ and the possible values for δ are : 1, 5, 6, 7, 8, 9, B or F. These values have also the same probability equal to $\frac{1}{8}$. The concerned trees are y_5^δ for every δ where $\alpha = (0100)_2$. The gain factor is:

$$\frac{\frac{1}{8}(16+4+4+4+16+16+16+4)}{16} = 2^{-0.68}$$

– For column 7, $\Delta = 4 \implies \delta \in \{5, 7, 12, 14\}$ and all optimal trees for y_4^δ are trivial trees. So there is no gain factor.

The total gain factor is: $2^{-1.87}$.

Column of type f: All other columns are of type f. Among them, we are only choosing those implying a gain of bits for the next round. They are two such columns.

– Column 0: when the difference is zero, we have a column of type c and we need to determine (y_0, y_1). We can absorb bits x_2 of columns 13 and 30. And when the difference is 1 then we have a column of type e and all trees for y_1^δ with $\alpha = 3$ are trivial trees but we can absorb x_2 of column 30. Therefore the gain factor is:

$$2^{-2} \cdot \frac{8}{16} + 2^{-1} = 2^{-0.68}$$

– Column 16: if the difference is zero then we have a column of type c for which the optimal tree has 12 leaves. When the difference is equal to 8 then all corresponding optimal trees are trivial. For both case, we can absorb bit x_1 of column 30 so the gain is:

$$\frac{12}{2 * 16} + 2^{-1} = 2^{-0.19}$$

The total gain factor of these columns is $2^{-0.87}$.

The total gain for the key-guessing strategy is $2^{-39.42}$.
In the upper extension, we need to guess 140 key bits including the columns of type a. The naive number of guesses, 2^{140}, is thus reduced to $2^{140-39.42}$.

Time complexity The cost of the distillation phase is $2^{-2.58}$. Specifically, the partial encryption is performed over two rounds, and the partial encryption of a plaintext can be reused for all input values of the inactive S-box. Next, a one-round decryption is performed. Finally, the filtering step requires 2^3 plaintexts to obtain one that satisfies the condition. Therefore, the overall cost is $(2^{-4} \cdot 2^3 \cdot 2/12 + 1/12) = 2^{-2.58}$.

The SERPENT block size is 128-bit, then $\rho_A \approx 256$ and $\rho_M \approx 128 \cdot log_2(128) \approx 961$. And a 12-round encryption in terms of bit operation is $\rho_E \approx 128 \cdot 3 \cdot 12 = 4608$ bit operations, with ρ_E the cost of one encryption. Taking into account the conditions described in the filtering step, $\kappa_l = \kappa_{in} + \kappa_{out} = 60$ bits where $\kappa_{in} = 4$. Then $\rho_M + \rho_A \kappa_{out} = 15297 = 2^{1.67} \rho_E$.

Δ_{in} : 0**0 **0* *00* *0** 0*** *0** *0** *00* // 21 active S-boxes #0

$After\ S_0$: 0**0 **05 *00* *0** 0*** *0** 40*1 *00* #1

$After\ LT$: 0$00 0000 0000 00c# 00!0 0d00 0000 0000 #2

$After\ S_1\ , 2^{-4}$: 0200 0000 0000 001a 00e0 0400 0000 0000 #3

$After\ LT\ , \Delta_i$: 0000 0000 0000 0000 0000 0000 0400 5000 #4

$After\ S_2\ , 2^{-5}$: 0000 0000 0000 0000 0000 0000 0a00 4000 #5

$After\ LT$: 0004 0000 0000 0000 0000 0000 0000 0000 #6

$After\ S_3\ , 2^{-3}$: 000a 0000 0000 0000 0000 0000 0000 0000 #7

$After\ LT$: 0020 0040 0000 0000 0000 0001 0000 8100 #8

$3\,tours\,milieu$: $2^{-7.38}$

$After\ LT\ , \Lambda_m$: 0000 0000 0001 0000 a000 0000 0000 0000 #9

$After\ S_7\ , 2^{-4}$: 0000 0000 0001 0000 1000 0000 0000 0000 #10

$After\ LT$: 000a 0000 0000 0000 0000 0000 0100 00b0 #11

$After\ S_0\ , 2^{-5}$: 0001 0000 0000 0000 0000 0000 0100 0010 #12

$After\ LT$: 0000 0001 0000 b000 0b00 00a0 0000 0000 #13

$After\ S_1\ , 2^{-6}$: 0000 0001 0000 1000 0100 0010 0000 0000 #14

$After\ LT$: 0000 b000 0a00 0000 0000 0100 00b0 000b #15

$After\ S_2\ , 2^{-4}$: 0000 1000 0100 0000 0000 0500 0010 000* #16

$After\ LT,\ \Lambda_o$: 000* 00*0 **00 0*00 00*0 000* ***0 *0** #17

$After\ S_3$: 000* 00*0 **00 0*00 00*0 000* ***0 *0** // 13 active S-boxes #18

Fig. 6: Extension of the new distinguisher. The truncated inputs in round #2 are such that $\$ = (0, -1, -1, -1), \# = (-1, 0, -1, -1)\ and\ ! = (-1, -1, -1, 0)$.

Differences in #1

	16	10		13			10							30	
	30	30		17	16			13			13				
				30			17	16			13			13	
	30			17	16			30			13	16		17	16

Values in #1

13_1	16_0	16_0		13_0	13_0	16_3			13_3		16_0	30_2			30_1
		30_1					30_0						16_2		
	α_0	30_2			30_1			16_2		30_0	α_1				16_2
								30_3				16_2	30_0		13_2
S_0^{31}	S_0^{30}	S_0^{29}	S_0^{28}	S_0^{27}	S_0^{26}	S_0^{25}	S_0^{24}	S_0^{23}	S_0^{22}	S_0^{21}	S_0^{20}	S_0^{19}	S_0^{18}	S_0^{17}	S_0^{16}

—

Differences in #1

											17	16			13
		13	13	16			13			10					
	17	16						10		30					
	16	10		13		16	30					10			

Values in #1

			30_0	30_0				30_3		16_2		16_2	α_5		13_2
α_2			13_1		16_0			13_0							30_2
α_3		13_2	α_4			13_1		16_0	16_3	30_2	13_0	13_3		16_0	
						16_3			13_3		16_0			α_6	
S_0^{15}	S_0^{14}	S_0^{13}	S_0^{12}	S_0^{11}	S_0^{10}	S_0^{9}	S_0^{8}	S_0^{7}	S_0^{6}	S_0^{5}	S_0^{4}	S_0^{3}	S_0^{2}	S_0^{1}	S_0^{0}

$\alpha_0 : 13_0, 30_2$ $\alpha_1 : 13_2, 30_3$ $\alpha_2 : 13_2, 16_1$ $\alpha_3 : 16_2, 30_0$ $\alpha_4 : 13_2, 16_1$ $\alpha_5 : 13_2, 16_1, 30_0$
$\alpha_6 : 13_0, 30_2$

		Gain
a	Free columns	2^{-4}
b	No difference : 1 value	$2^{-13.49}$
c	No difference : 2 values	$2^{-4.68}$
d	Variable difference : No value	$2^{-14.51}$
e	Fixed difference : 1 to 3 values	$2^{-1.87}$
f	variable difference : 1 to 3 values	$2^{-0.87}$
		$2^{-39.42}$

Fig. 7: Columns of the S-boxes S_0 in round #1. See Section 2.3.2 for the definition of a column. Each cell contains only the S-box index: for example, S_r^j is written as j, meaning the j-th S-box in round r, and $S_{r+1}^{m,q}$ is written as m_q, representing the q-th bit difference (Δx_q) of the m-th S-box in the next round. The colors indicate the column types (see the legend). In the differences, a gray cell indicates that its value is fixed (equal to 1).

The key recovery cost is: $2^{140-39.42}(2^{115.5-2.58}+2\cdot 2^{116+1.67}) \approx 2^{219.27}$ encryptions. The advantage a is 38, so the cost of the exhaustive search is 2^{256-38}. So the time complexity of this attack with a complete exhaustive search is $2^{219.77}$ encryptions.

For the case when the success probability is 0.85, the time complexity is $\approx 2^{220.79}$ encryptions.

Toward a better data complexity To achieve a better data complexity, we can fix one more transition and abandon conditions we defined before so that no sieving is performed. By adding the column 17 among the fixed transition in #2, the state of S_0 changes as shown in Figure 11 of Appendix C. We performed the same computations as previously and obtained a gain of $2^{-16.78}$. We also removed conditions in the linear part, therefore the number of active S-boxes is 13 (i.e $\kappa_{out} = 52$). The resulting data complexity is $2^{110.41}$ with an advantage of 20 guessed key bits, a probability of success of 0.1 and a time complexity equal to:

$$2^{144-16.78}(2^{110.41-2.58} + 2 \cdot 2^{108+1.57}) + 2^{256-20} \approx 2^{238.38} \text{ encryptions.}$$

4.2 PRESENT

In this section, we apply our tool to the block cipher PRESENT [BKL+07], developed in 2007. To the best of our knowledge, this is the first differential-linear key recovery attacks on this cipher, however the best attacks are linear attacks. Therefore, the application to PRESENT aims to prove that our tool find distinguishers that are more suited for key recovery attacks that those found in [HDE24] (cf. Table 4).

Description of PRESENT PRESENT is a key-alternating block cipher with an internal state of 64 bits: $x = x_{63} \dots x_0$, with x_0 being the least significant bit (LSB). The key size is either 80 or 128 bits, and we denote it by PRESENT-κ when the key size is κ. The encryption consists of the successive application of a round function, which includes: a key addition **AddRoundKey**, a substitution layer $\mathcal{S}$, and a bitwise permutation $\mathcal{P}$.

- **AddRoundKey**: Each round i uses a 64-bit subkey k_i, generated by the key schedule algorithm. The only difference between PRESENT-80 and PRESENT-128 lies in this key schedule. The **AddRoundKey** operation consists of a bitwise XOR between the subkey k_i and the state x_i.
- **S-box layer** $\mathcal{S}$: This layer applies the following 4×4 S-box S in parallel to the sixteen 4-bit nibbles of the state. Specifically, for $j = 0 \dots 15$, the S-box is applied to $x_{4j+3}x_{4j+2}x_{4j+1}x_{4j}$:

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

- **Permutation** $\mathcal{P}$: A bitwise permutation is applied to the state. It is defined as follows:

$$\begin{aligned} \mathcal{P} : \{0, \dots, 63\} &\longrightarrow \{0, \dots, 63\} \\ \mathcal{P}(j) &:= 16j \bmod 63, \quad \text{for } j \neq 63 \\ \mathcal{P}(63) &:= 63 \end{aligned}$$

Attacks of PRESENT For PRESENT too, extending directly the distinguishers found in [HDE24] does not yield to viable attacks up to a certain number of rounds. Precisely, the distinguisher in [HDE24], can yield attacks that reaches at most 15 rounds for PRESENT-80 and 16 rounds for PRESENT-128. However, our tool were able to find a distinguisher reaching one additional round for PRESENT-80 and two more rounds for PRESENT-128.

Running our tool for a 16-round key recovery attack directly produces a viable attack, including its estimated data and time complexities, as well as a distinguisher with a correlation of $2^{-28.86}$. Therefore, there is no need to explore other options for the middle part. However, when running it for a 17-round key recovery attack, the tool did not directly yield a viable solution due to a correlation that was too low in the middle part of the cipher. Consequently, the model proposed alternative candidates—using our rotation-based approach described in Subsection 3.3 with potentially higher correlations. Among these, we identified a suitable middle part, which led to a distinguisher with the characteristics indicated in Table 7 and with $r_0 = 1, r_m = 8$ and $r_1 = 4$.

Table 7: DL distinguisher characteristics for 13-round PRESENT.

$r_0 = 1, r_m = 8, r_1 = 4$	
$p = 2^{-4}, r = 2^{-7.72}, q^2 = 2^{-16}$	$\epsilon = 2^{-27.72}$
$\Delta_i = 0x0000000090090000$	$\Delta_m = 0x0000009000000000$
$\Lambda_m = 0x0000000000200000$	$\Lambda_o = 0x8000000000000000$

The distinguisher has a higher correlation than the one initially proposed for the 16 rounds and by the way gives a better attack on 16 rounds of PRESENT-80. This distinguisher also gives an 18-round attack on PRESENT-128 which is better than the 17 expected.

This distinguisher (without extended rounds) is drawn in Figure 17 of Appendix E.

16 rounds attack of PRESENT-80. The distinguisher with extended rounds is represented in Figure 8.

The overall correlation of the distinguisher remains $2^{-27.72}$. Using the same model as previously, our key recovery attack requires approximately $N \approx 2^{57.88}$ pairs with a success probability Ps of 0.1 and an advantage a of 15 bits of key guesses. With the same advantage, we can also reach a success probability Ps of 0.825, resulting in a worse-case data complexity of $2^{63.91}$.

For the serpent block cipher, $\rho_A \approx 128, \rho_M \approx 3 \cdot 64^{log_2(3)} \approx 2143$, $\rho_E \approx 2 \cdot 64 \cdot r + 64$ and $\kappa_{out} = 16$ For $r = 13, \rho_E \approx 1728 \implies \rho_M + \rho_A \kappa_{out} \approx 2^{1.28} \rho_E$.

The time complexity is then the sum of the key recovery cost and the cost of exhaustive search. For a success probability Ps of 0.1, the time complexity is:

$$2^{4 \cdot 4} \cdot (2^{57.88} + 2^{20+2.28}) + 2^{80-15} \approx 2^{73.88} \text{ encryptions.}$$

$$
\begin{aligned}
\Delta: &\ *\,0\,0\,*\,0\,0\,0\,0\,0\,0\,0\,0\,*\,0\,0\,* & \\
output, \mathcal{S}: &\ 2\,0\,0\,2\,0\,0\,0\,0\,0\,0\,0\,0\,2\,0\,0\,2 & \#0 \\
After, \mathcal{P}, \Delta_i: &\ 0\,0\,0\,0\,0\,0\,0\,0\,9\,0\,0\,9\,0\,0\,0\,0 & \#1 \\
13 \text{ rounds}: &\ 2^{-27.72} & \\
After, \mathcal{P}, \Lambda_o: &\ 8\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0 & \#3 \\
Output, S, q=1: &\ *\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0 & \#4 \\
After, \mathcal{P}: &\ *\,0\,0\,0\,*\,0\,0\,0\,*\,0\,0\,0\,*\,0\,0\,0 & \#5 \\
Output, S: &\ *\,0\,0\,0\,*\,0\,0\,0\,*\,0\,0\,0\,*\,0\,0\,0 & \#6 \\
\Lambda: &\ *\,0\,0\,0\,*\,0\,0\,0\,*\,0\,0\,0\,*\,0\,0\,0 &
\end{aligned}
$$

Fig. 8: Extended distinguisher for the key recovery attack on 16-round of PRESENT-80.

When the probability Ps is 0.825, the time complexity becomes $\approx 2^{79.91}$ encryptions.

The attack on 18 rounds of PRESENT-128. To achieve 18 rounds, we extended the differential part by two rounds and the linear part by three rounds, as shown in Figure 9.

The extended rounds in the linear part contribute an additional correlation of 2^{-2}, resulting in an overall distinguisher correlation of $2^{-27.72-2\times 2} = 2^{-31.72}$. With this correlation, the best achievable advantage is 4, with probability of success equal to 0.1. The corresponding number of plaintext/ciphertext pairs required for the attack is approximately $N \approx 2^{63.25}$. Therefore, the time complexity is given by: $2^{4\cdot 12} \cdot (2^{63.25} + 2^{20+2.28}) + 2^{128-4} \approx 2^{124}$ encryptions.

5 Conclusion

In this paper, we presented a method to automatically extend the tool from [HDE24] into a key recovery tool, as directly extending their distinguishers does not yield viable attacks beyond a certain number of rounds. Our approach enables the identification of slightly weaker distinguishers that are nonetheless better suited for key recovery, ultimately leading to the successful attacks described in this work.

A key limitation of the current tool lies in its treatment of the middle-part correlation, which is not directly incorporated into the objective function. To mitigate this, we introduced an ad-hoc method in an auxiliary module to search for improved middle-part characteristics from the CP program output.

The gain factors are not yet automatically obtained from the proposed tool, as the tool currently only searches for a trade-off between the number of active

$$
\begin{aligned}
\Delta: &\ * * * * 0\,0\,0\,0\,0\,0\,0\,0 * * * * && \\
Output, S: &\ * * * * 0\,0\,0\,0\,0\,0\,0\,0 * * * * && \#0 \\
After, \mathcal{P}: &\ * \,0\,0\, * \,0\,0\,0\,0\,0\,0\,0\, * \,0\,0\, * && \#1 \\
Output, S, p=1: &\ 2\,0\,0\,2\,0\,0\,0\,0\,0\,0\,0\,0\,2\,0\,0\,2 && \#2 \\
After, \mathcal{P}, \Delta_i: &\ 0\,0\,0\,0\,0\,0\,0\,0\,9\,0\,0\,9\,0\,0\,0\,0 && \#3 \\
13 \text{ rounds}: &\ 2^{-27.72} && \\
After, \mathcal{P}, \Lambda_o: &\ 8\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0 && \#4 \\
Output, S, q=2^{-2}: &\ 2\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0 && \#5 \\
After, \mathcal{P}: &\ 0\,0\,0\,0\,0\,0\,0\,0\,8\,0\,0\,0\,0\,0\,0\,0 && \#6 \\
Output, S, q=1: &\ 0\,0\,0\,0\,0\,0\,0\,0 * 0\,0\,0\,0\,0\,0\,0 && \#7 \\
After, \mathcal{P}: &\ 0 * 0\,0\,0 * 0\,0\,0 * 0\,0\,0 * 0\,0 && \#8 \\
Output, S: &\ 0 * 0\,0\,0 * 0\,0\,0 * 0\,0\,0 * 0\,0 && \#9 \\
\Lambda: &\ 0 * 0\,0\,0 * 0\,0\,0 * 0\,0\,0 * 0\,0 &&
\end{aligned}
$$

Fig. 9: Extended distinguisher for the key recovery attack on 18-round of PRESENT-128.

bits and the probability of the distinguisher. However, the columns are related to the states of the S-boxes in the extended rounds, and by minimizing the number of active S-boxes, we implicitly maximize the presence of columns of type a/b/c, which in turn provides a good gain factor.

For future work, it would be interesting to investigate ways to compute the middle-part correlation directly within the CP model. Moreover, integrating the key-guessing framework with the data-complexity analysis framework would allow the model to directly estimate the full attack complexity, providing a more comprehensive and automated solution.

Acknowledgment

The authors wish to thank V. Lallemand for her helpful discussions and valuable insights during the early stages of this work, as well as the anonymous reviewers for their constructive and detailed comments that significantly improved the quality of this paper. This work has been partially supported by the France 2030 program under grant agreement No. ANR-22-PECY-0010 CRYPTANALYSE.

Bibliography

[BAK98] Eli Biham, Ross J. Anderson, and Lars R. Knudsen. Serpent: A new block cipher proposal. In Serge Vaudenay, editor, *FSE'98*, volume 1372 of *LNCS*, pages 222–238. Springer, Berlin, Heidelberg, March 1998.

[BCD+22] Marek Broll, Federico Canale, Nicolas David, Antonio Flórez-Gutiérrez, Gregor Leander, María Naya-Plasencia, and Yosuke Todo. New attacks from old distinguishers improved attacks on Serpent. In Steven D. Galbraith, editor, *CT-RSA 2022*, volume 13161 of *LNCS*, pages 484–510. Springer, Cham, March 2022.

[BCFG+21] Marek Broll, Federico Canale, Antonio Flórez-Gutiérrez, Gregor Leander, and María Naya-Plasencia. Generic framework for key-guessing improvements. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 453–483. Springer, Cham, December 2021.

[BDD+24] Christina Boura, Nicolas David, Patrick Derbez, Rachelle Heim Boissier, and María Naya-Plasencia. A generic algorithm for efficient key recovery in differential attacks - and its associated tool. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 217–248. Springer, Cham, May 2024.

[BDK01] Eli Biham, Orr Dunkelman, and Nathan Keller. The rectangle attack - rectangling the Serpent. In Birgit Pfitzmann, editor, *EUROCRYPT 2001*, volume 2045 of *LNCS*, pages 340–357. Springer, Berlin, Heidelberg, May 2001.

[BDK02a] Eli Biham, Orr Dunkelman, and Nathan Keller. Linear cryptanalysis of reduced round Serpent. In Mitsuru Matsui, editor, *FSE 2001*, volume 2355 of *LNCS*, pages 16–27. Springer, Berlin, Heidelberg, April 2002.

[BDK02b] Eli Biham, Orr Dunkelman, and Nathan Keller. New results on boomerang and rectangle attacks. In Joan Daemen and Vincent Rijmen, editors, *FSE 2002*, volume 2365 of *LNCS*, pages 1–16. Springer, Berlin, Heidelberg, February 2002.

[BDK03] Eli Biham, Orr Dunkelman, and Nathan Keller. Differential-linear cryptanalysis of Serpent. In Thomas Johansson, editor, *FSE 2003*, volume 2887 of *LNCS*, pages 9–21. Springer, Berlin, Heidelberg, February 2003.

[BDKW19] Achiya Bar-On, Orr Dunkelman, Nathan Keller, and Ariel Weizman. DLCT: A new tool for differential-linear cryptanalysis. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 313–342. Springer, Cham, May 2019.

[BKL+07] Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin,

and C. Vikkelsoe. PRESENT: An ultra-lightweight block cipher. In Pascal Paillier and Ingrid Verbauwhede, editors, *CHES 2007*, volume 4727 of *LNCS*, pages 450–466. Springer, Berlin, Heidelberg, September 2007.

[BLN15] Céline Blondeau, Gregor Leander, and Kaisa Nyberg. Differential-linear cryptanalysis revisited. In Carlos Cid and Christian Rechberger, editors, *FSE 2014*, volume 8540 of *LNCS*, pages 411–430. Springer, Berlin, Heidelberg, March 2015.

[BN16] Céline Blondeau and Kaisa Nyberg. Improved parameter estimates for correlation and capacity deviates in linear cryptanalysis. *IACR Trans. Symm. Cryptol.*, 2016(2):162–191, 2016.

[BS91] Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, 4(1):3–72, January 1991.

[CSQ07] Baudoin Collard, François-Xavier Standaert, and Jean-Jacques Quisquater. Improving the time complexity of Matsui's linear cryptanalysis. In Kil-Hyun Nam and Gwangsoo Rhee, editors, *ICISC 07*, volume 4817 of *LNCS*, pages 77–88. Springer, Berlin, Heidelberg, November 2007.

[CSQ08] Baudoin Collard, François-Xavier Standaert, and Jean-Jacques Quisquater. Experiments on the multiple linear cryptanalysis of reduced round Serpent. In Kaisa Nyberg, editor, *FSE 2008*, volume 5086 of *LNCS*, pages 382–397. Springer, Berlin, Heidelberg, February 2008.

[DIK08] Orr Dunkelman, Sebastiaan Indesteege, and Nathan Keller. A differential-linear attack on 12-round Serpent. In Dipanwita Roy Chowdhury, Vincent Rijmen, and Abhijit Das, editors, *INDOCRYPT 2008*, volume 5365 of *LNCS*, pages 308–321. Springer, Berlin, Heidelberg, December 2008.

[FGT24] Antonio Flórez-Gutiérrez and Yosuke Todo. Improving linear key recovery attacks using walsh spectrum puncturing. Cryptology ePrint Archive, Report 2024/151, 2024.

[FN20] Antonio Flórez-Gutiérrez and María Naya-Plasencia. Improving key-recovery in linear attacks: Application to 28-round PRESENT. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 221–249. Springer, Cham, May 2020.

[HCN08] Miia Hermelin, Joo Yeon Cho, and Kaisa Nyberg. Multidimensional linear cryptanalysis of reduced round Serpent. In Yi Mu, Willy Susilo, and Jennifer Seberry, editors, *ACISP 08*, volume 5107 of *LNCS*, pages 203–215. Springer, Berlin, Heidelberg, July 2008.

[HDE24] Hosein Hadipour, Patrick Derbez, and Maria Eichlseder. Revisiting differential-linear attacks via a boomerang perspective with application to AES, ascon, CLEFIA, SKINNY, PRESENT, KNOT, TWINE, WARP, LBlock, simeck, and SERPENT. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IV*, volume 14923 of *LNCS*, pages 38–72. Springer, Cham, August 2024.

[HNE22] Hosein Hadipour, Marcel Nageler, and Maria Eichlseder. Throwing boomerangs into Feistel structures application to CLEFIA, WARP, LBlock, LBlock-s and TWINE. *IACR Trans. Symm. Cryptol.*, 2022(3):271–302, 2022.

[KKS01] John Kelsey, Tadayoshi Kohno, and Bruce Schneier. Amplified boomerang attacks against reduced-round MARS and Serpent. In Bruce Schneier, editor, *FSE 2000*, volume 1978 of *LNCS*, pages 75–93. Springer, Berlin, Heidelberg, April 2001.

[LH94] Susan K. Langford and Martin E. Hellman. Differential-linear cryptanalysis. In Yvo Desmedt, editor, *CRYPTO'94*, volume 839 of *LNCS*, pages 17–25. Springer, Berlin, Heidelberg, August 1994.

[LLL21] Meicheng Liu, Xiaojuan Lu, and Dongdai Lin. Differential-linear cryptanalysis from an algebraic perspective. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 247–277, Virtual Event, August 2021. Springer, Cham.

[Mat94] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In Tor Helleseth, editor, *EUROCRYPT'93*, volume 765 of *LNCS*, pages 386–397. Springer, Berlin, Heidelberg, May 1994.

[MC13] James McLaughlin and John A. Clark. Filtered nonlinear cryptanalysis of reduced-round Serpent, and the wrong-key randomization hypothesis. In Martijn Stam, editor, *14th IMA International Conference on Cryptography and Coding*, volume 8308 of *LNCS*, pages 120–140. Springer, Berlin, Heidelberg, December 2013.

[NWW11] Phuong Ha Nguyen, Hongjun Wu, and Huaxiong Wang. Improving the algorithm 2 in multidimensional linear cryptanalysis. In Udaya Parampalli and Philip Hawkes, editors, *ACISP 11*, volume 6812 of *LNCS*, pages 61–74. Springer, Berlin, Heidelberg, July 2011.

[WLW24] Wenhui Wu, Muzhou Li, and Meiqin Wang. Improved linear key recovery attacks on PRESENT. *IEEE Trans. Inf. Theory*, 70(12):9195–9213, 2024.

Appendix

A Old distinguisher of SERPENT and its extension from [BCD+22]

In this representation, an S-box in the extended rounds is considered active if its output difference is non-zero or if one of its output values is required. (If, instead, we define an S-box as active only when its output difference is non-zero, then states #0 and #1 would be: **0* *0** 0*** 0*** *0** *0** **** 00**)

Δ_{in} :	**0* **** 0*** **** **** **** **** ****	// 30 active S-boxes #0
$After\ S_0$:	**0* **** 0*** **** **** **** **** ****	#1
$After\ LT$:	@000 0000 0000 0$#0 0800 9000 0000 0000	#2
$After\ S_1\ , 2^{-4}$:	2000 0000 0000 01a0 0e00 4000 0000 0000	#3
$After\ LT\ , \Delta_i$:	0000 0000 0000 0000 0000 0000 4005 0000	#4
$After\ S_2\ , 2^{-5}$:	0000 0000 0000 0000 0000 0000 a004 0000	#5
$After\ LT$:	0040 0000 0000 0000 0000 0000 0000 0000	#6
$After\ S_3\ , 2^{-1}$:	00*0 0000 0000 0000 0000 0000 0000 0000	#7
$After\ LT$:	0??0 0?00 0?00 0000 000? 00?0 ??0? ?0?0	#8
$After\ S_4$:	0??0 0?00 0?00 0000 000? 00?0 ??0? ?0?0	#9
$After\ LT$:	0020 0000 0000 0000 0000 0000 0000 0002	#10
$After\ S_5\ , 2^{-4}$:	0040 0000 0000 0000 0000 0000 0000 0008	#11
$After\ LT$:	0000 0000 0000 0000 0000 0000 8000 0000	#12
$After\ S_6\ , 2^{-2}$:	0000 0000 0000 0000 0000 0000 1000 0000	#13
$After\ LT$:	0000 00a0 0001 0000 0000 0000 0000 0000	#14
$After\ S_7\ , 2^{-4}$:	0000 0010 0001 0000 0000 0000 0000 0000	#15
$After\ LT$:	0000 0000 0000 0000 0000 1000 0b00 00a0	#16
$After\ S_0\ , 2^{-5}$:	0000 0000 0000 0000 0000 1000 0100 0010	#17
$After\ LT$:	0010 000b 0000 b000 0a00 0000 0000 0000	#16
$After\ S_1\ , 2^{-6}$:	0010 0001 0000 1000 0100 0000 0000 0000	#17
$After\ LT$:	0000 a000 0000 0000 1000 0b00 00b0 000b	#16
$After\ S_2\ , 2^{-4}$:	0000 1000 0000 0000 5000 0100 0010 000*	#17
$After\ LT$:	000* 00*0 **00 0*00 00*0 *00* *0*0 *0**	#18
$After\ S_3$:	000* 00*0 **00 0*00 00*0 *00* *0*0 *0**	// 13 active S-boxes

Fig. 10: Old distinguisher extension from [BCD+22]. Truncated input of S_1 in round #2 are $@ = (0, -1, -1, -1), \$ = (-1, -1, -1, 1)\ and\ \# = (-1, -1, -1, 0)$.

B Algorithm required for the Probabilistic Exension, Key Recovery and the Conditional Cryptanalysis

Algorithm 1 Constraints for the non-deterministic strategy. Evaluation of $S_0^{-1}(a_0, a_1, a_2, a_3) = (b_0, b_1, b_2, b_3)$. // a_0, b_0 are the LSBs

Require: (a_0, a_1, a_2, a_3)

if $a_0 == 0 \wedge a_1 == 0 \wedge a_2 == 0 \wedge a_3 == 0$ **then**
    $((b_0 = 0 \wedge b_1 = 0 \wedge b_2 = 0 \wedge b_3 = 0) \wedge (\mathfrak{p}_0 + \mathfrak{p}_1 = 0))$
else if $a_0 == 0 \wedge a_1 == 1 \wedge a_2 == 0 \wedge a_3 == 0$ **then**
    $((b_0 = 1 \wedge b_1 = -1 \wedge b_2 = -1 \wedge b_3 = -1) \wedge (\mathfrak{p}_0 + \mathfrak{p}_1 = 0))$
    $\vee$
    sbox_diff$(a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, \mathfrak{p}_0, \mathfrak{p}_1)$
else if $a_0 == 1 \wedge a_1 == 0 \wedge a_2 == 0 \wedge a_3 == 0$ **then**
    $((b_0 = -1 \wedge b_1 = 1 \wedge b_2 = -1 \wedge b_3 = -1) \wedge (\mathfrak{p}_0 + \mathfrak{p}_1 = 0))$
    $\vee$
    sbox_diff$(a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, \mathfrak{p}_0, \mathfrak{p}_1)$
else if $a_0 == 1 \wedge a_1 == 1 \wedge a_2 == 0 \wedge a_3 == 1$ **then**
    $((b_0 = -1 \wedge b_1 = 0 \wedge b_2 = -1 \wedge b_3 = -1) \wedge (\mathfrak{p}_0 + \mathfrak{p}_1 = 0))$
    $\vee$
    sbox_diff$(a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, \mathfrak{p}_0, \mathfrak{p}_1)$
else
    fixed_transition$(a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3) \wedge (\mathfrak{p}_0 + \mathfrak{p}_1 = 0)$
    $\vee$
    sbox_diff$(a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, \mathfrak{p}_0, \mathfrak{p}_1)$
end if

▷ The probabilistic model (sbox_diff) gives the probability $p_u = 2^{-\mathfrak{p}_u}$ of the transition where $\mathfrak{p}_u$ is a linear combination of $\mathfrak{p}_0$ and $\mathfrak{p}_1$. the coefficient of this linear combination are determined by the model of the DDT provided by the s-box analyzer tool [HNE22]. In the case of the SERPENT or PRESENT S-boxes, the DDT shows that the only possible probabilities are 1 (trivial case), 2^{-3}, or 2^{-2}. For their modeling, one can introduce the binary variables $\mathfrak{p}_0$ and $\mathfrak{p}_1$, where $\mathfrak{p}_0 = 1$ if the probability is 2^{-3} (and 0 otherwise), and $\mathfrak{p}_1 = 1$ if the probability is 2^{-2} (and 0 otherwise). Then, if the S-box is crossed with probability $p = 2^{-\mathfrak{p}_u}$, set $\mathfrak{p}_u = 3\mathfrak{p}_0 + 2\mathfrak{p}_1$.

Algorithm 2 Differential-Linear Key Recovery

Require: $N, \kappa_u, \kappa_l, \Delta_u, \Delta_i \, et \, \lambda_o$
Ensure: $\kappa_u + \kappa_l$ bits of the key K.
Choose N pairs $(P, C) \in \mathcal{P} \times \mathcal{C}$ ▷ $\mathcal{P}$ is the plaintexts space and $\mathcal{C}$ is the ciphertexts space
$\mathcal{P}_{out} \leftarrow \emptyset$
for sk_u **do** ▷ each subkeys of the differential part of size κ_u
    $cpt[1..2^{\kappa_l}] = [0..0]$ ▷ initialize 2^{κ_l} counters
    for $P \in \mathcal{P}$ **do**
        Partially encrypt P and $P \oplus \Delta_u$ into $X \, and \, X'$
        if $X \oplus X' == \Delta_i$ **then**
            $\mathcal{P}_{out} = \mathcal{P}_{out} \cup \{P\}$ ▷ keeps only pairs entering the distinguisher
        end if
    end for
    $csk^l \leftarrow 0$ ▷ candidate subkey for linear part
    for sk_l **do** ▷ each possible subkey in linear part
        for $P_o \in \mathcal{P}_{out}$ **do**
            Partially decrypt C_o encryption of P_o into Y_o
            Partially decrypt C'_o encryption of $P_o \oplus \Delta_i$ int Y'_o
            if $\lambda_o \cdot Y_o == \lambda_o \cdot Y'_o$ **then**
                $cpt[sk_l]$++
            end if
        end for
        if $cpt[sk_l] > cpt[csk^l]$ **then**
            $csk^l \leftarrow sk_l$
        end if
    end for
end for
return The most frequent (sk_u, csk^l)

Algorithm 3 Conditional cryptanalysis: pairs construction

.

Choose a plaintext x
Guess the 128 bits of k_0 and 12 bits of k_1
Compute $z = S_1 \circ LT \circ S_0(x)$ ▷ Gives $z[13], z[16] \, et \, z[30]$.
Compute $\Delta[30] = S_1^{-1}(z[30]+2)\,, \Delta[16] = S_1^{-1}(z[16]+A) \, et \, \Delta[13] = S_1^{-1}(z[13]+E)$
if $\Delta[30]_0 = 1 \vee \Delta[16]_1 = 1 \vee \Delta[13]_3 = 1$ **then**
    Reject x
    Go to step 1
end if
Let $\Delta[10] = 4$, $\Delta[17] = 1$ and $\Delta[i] = 0\,, \forall i \notin \{10, 13, 16, 17, 30\}$ ▷ Giving therefore the right input difference Δ of S_1
The plaintext x' is such that $x' = S_0^{-1}(S_0(x + k_0) + LT^{-1}(\Delta)) + k_0$
return (x, x')

C State of S_0 in #2 with 4 fixed transitions

Differences in #1

17	16	10		13			10							30	
	30	30		17	16	30	17	13			13			17	17
				30			17	16		17	13			13	
	30		30	17	16		17	30			13	16		17	16

Values in #1

α_0	α_1	16_0		13_0	α_3	16_3			13_3	17_0	16_0	30_2			30_1
		30_1					30_0					17_2	16_2		17_1
	α_2	30_2			30_1		17_2	16_2		30_0	α_4			17_2	α_5
								30_3			17_2	16_2	30_0		13_2
S_0^{31}	S_0^{30}	S_0^{29}	S_0^{28}	S_0^{27}	S_0^{26}	S_0^{25}	S_0^{24}	S_0^{23}	S_0^{22}	S_0^{21}	S_0^{20}	S_0^{19}	S_0^{18}	S_0^{17}	S_0^{16}

—

Differences in #1

											17	16			13
16		13	13	16			13			10					
	17	16			13			10		30					
17	16	13		13	17	16	30		13			10			

Values in #1

			30_0	30_0				30_3	17_2	16_2	17_2	α_{11}	α_{12}		13_2
α_6			13_1	17_0	16_0			13_0							30_2
α_7		α_8	α_9			13_1	17_0	α_{10}	16_3	30_2	13_0	13_3	17_0	16_0	
					17_3	16_3			13_3	17_0	16_0			α_{13}	
S_0^{15}	S_0^{14}	S_0^{13}	S_0^{12}	S_0^{11}	S_0^{10}	S_0^9	S_0^8	S_0^7	S_0^6	S_0^5	S_0^4	S_0^3	S_0^2	S_0^1	S_0^0

$\alpha_0 : 13_1, 17_0$ $\alpha_1 : 16_0, 17_0$ $\alpha_2 : 13_0, 30_2$ $\alpha_3 : 13_0, 17_3$ $\alpha_4 : 13_2, 30_3$ $\alpha_5 : 16_2, 17_2$
$\alpha_6 : 13_2, 16_1$ $\alpha_7 : 16_2, 30_0$ $\alpha_8 : 13_2, 17_1$ $\alpha_9 : 13_2, 16_1$ $\alpha_{10} : 16_0, 17_3$ $\alpha_{11} : 16_2, 17_1$
$\alpha_{12} : 13_2, 16_1, 30_0$ $\alpha_{13} : 13_0, 30_2$

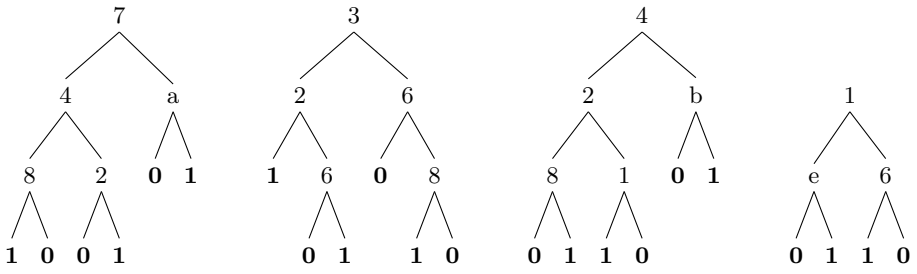
		Gain
a	Free columns	2^0
b	No difference : 1 value	$2^{-2.41}$
c	No difference : 2 values	$2^{-6.68}$
d	Variable difference : No value	$2^{-2.36}$
e	Fixed difference : 1 to 3 values	2^0
f	variable difference : 1 to 3 values	$2^{-5.33}$
		$2^{-16.78}$

Fig. 11: State after in #1 of S_0.

D Trees

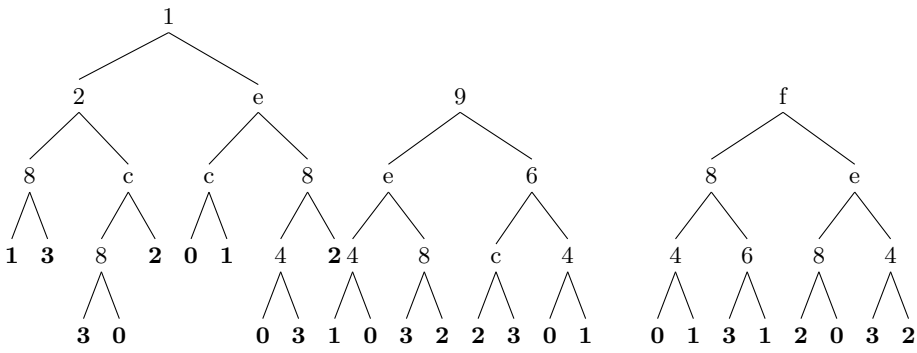
We construct theses trees using the conditional library provided in [BCFG+21].

Column	a	b	c	d	e	f
Trees figure	-	12	13	14	15	-



(a) y_0 (b) y_1 (c) y_2 (d) y_3

Fig. 12: Columns of type b: function $< 2^i, S(x) >$.

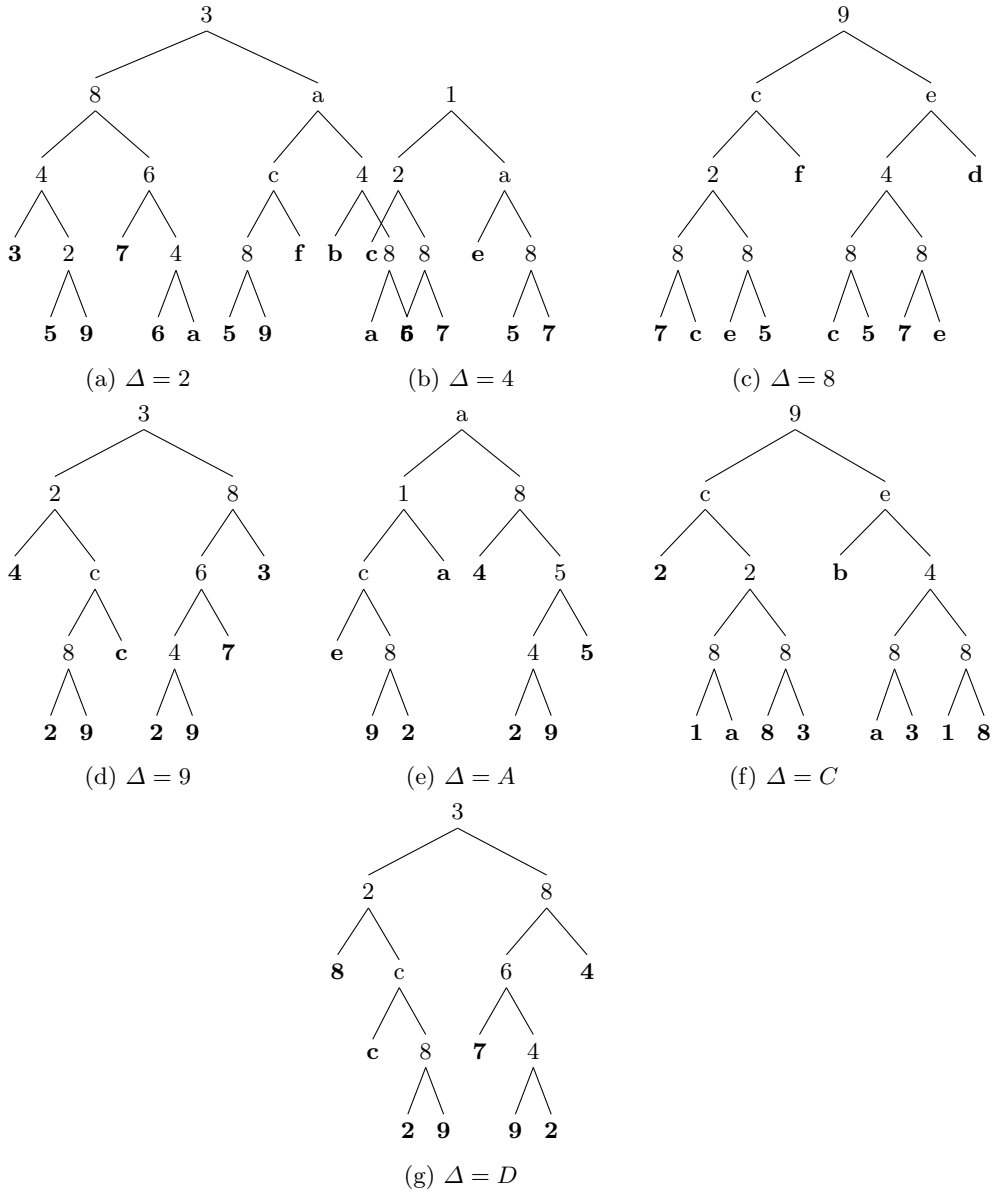


(a) (y_1, y_2) (b) (y_1, y_3) (c) (y_2, y_3)

Fig. 13: Columns of type c: function $< 2^i + 2^j, S(x) >, i \neq j$ and $i, j \in \{0, 1, 2, 3\}$.

E Distinguishers

- SERPENT distinguisher is represented in figure 16.
- PRESENT distinguisher is represented in figure 17.

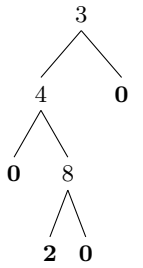


(a) $\Delta = 2$ (b) $\Delta = 4$ (c) $\Delta = 8$

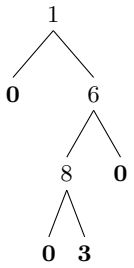
(d) $\Delta = 9$ (e) $\Delta = A$ (f) $\Delta = C$

(g) $\Delta = D$

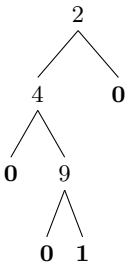
Fig. 14: Columns of type d: function $F_\Delta(x) = S^{-1}(S(x) + \Delta) + x$.



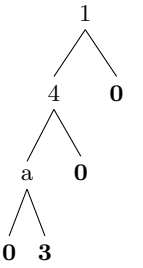
(a) $\Delta = 1, \delta = 3, \alpha = (0011)_2$



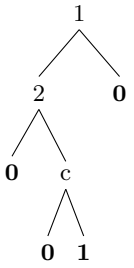
(b) $\Delta = 1, \delta = 6, \alpha = (0011)_2$



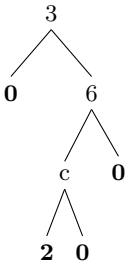
(c) $\Delta = 1, \delta = 9, \alpha = (0011)_2$



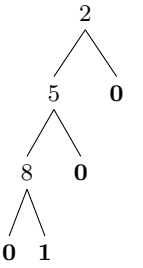
(d) $\Delta = 1, \delta = A, \alpha = (0011)_2$



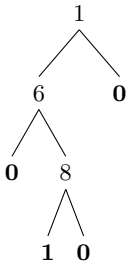
(e) $\Delta = 1, \delta = C, \alpha = (0011)_2$



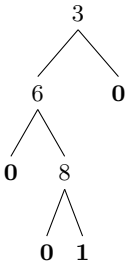
(f) $\Delta = 1, \delta = f, \alpha = (0011)_2$



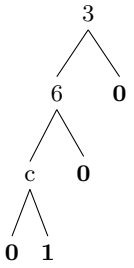
(g) $\Delta = 5, \delta = 5, \alpha = (0100)_2$



(h) $\Delta = 5, \delta = 6, \alpha = (0100)_2$



(i) $\Delta = 5, \delta = 7, \alpha = (0100)_2$



(j) $\Delta = 5, \delta = f, \alpha = (0100)_2$

Fig. 15: Columns of type e: function $y^{\delta}_{\Delta}(x) =< 1 - g^{\delta}_{\Delta}(x)) \cdot < \alpha, S(x+\delta) + \Delta >$.

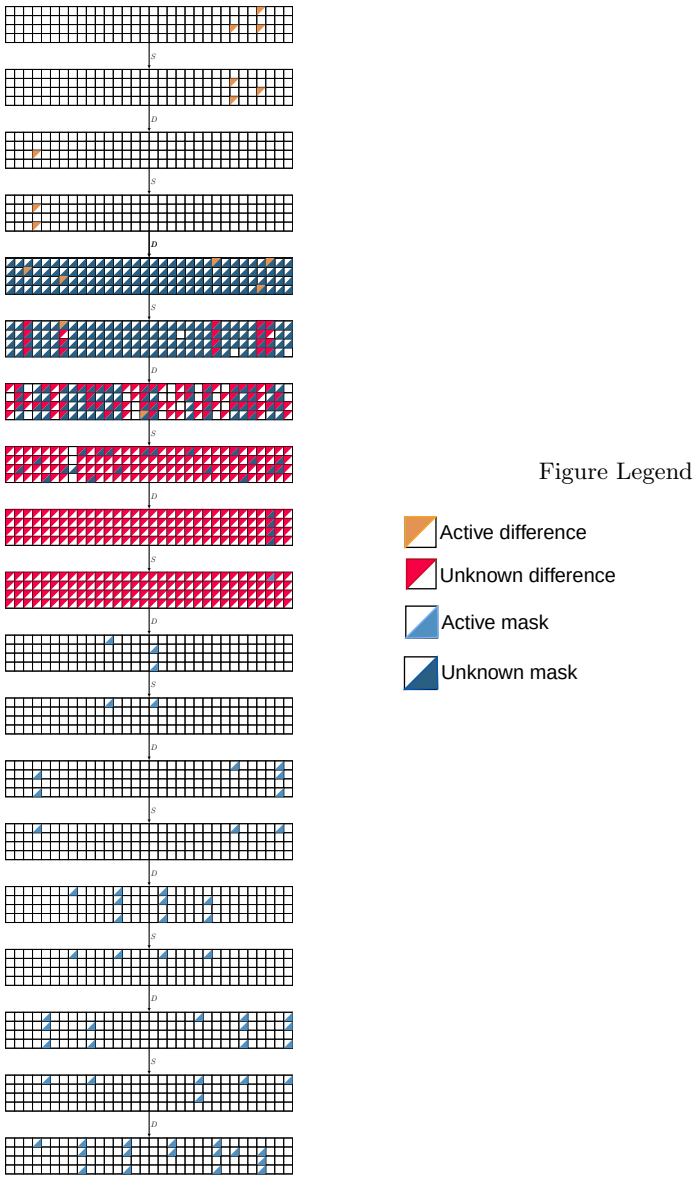


Fig. 16: SERPENT 9 rounds DL distinguisher.

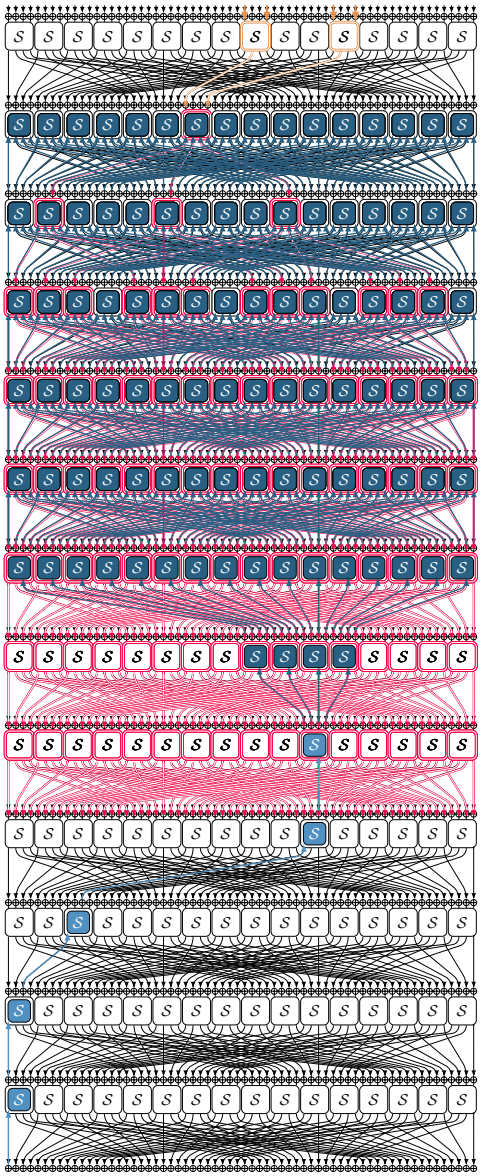


Fig. 17: 13 rounds DL distinguisher of PRESENT. (– Differential, – Linear)